
DOSSIER DE PROJET



GESTION D'UN CHAMPIONNAT DE COURSES EN LIGNE

Racing Championship App – Web MVC PHP platform

ACHARD Cédric

CEFii Angers

Titre professionnel Développeur Web et Web Mobile

Projet réalisé en autonomie dans le cadre de l'examen

Novembre – Décembre 2025

Table des matières

I. Introduction avec la liste des compétences du référentiel couvertes par le projet ...	1
1.1 Développer la partie front-end d'une application web ou web mobile sécurisée.....	1
1.2 Développer la partie back-end d'une application web ou web mobile sécurisée	2
II. Contexte du projet	3
2.1 Présentation personnelle et de la communauté	3
2.2 Cahier des charges et expression des besoins.....	3
2.3 Contraintes du projet et livrables attendus.....	4
2.4 Environnement humain et technique	5
III. Réalisations comportant les éléments les plus significatifs côté front-end, les arguments ayant conduit à ces choix, y compris pour la sécurité	6
3.1 Présentation de maquettes de l'application, adaptation web et adaptation web mobile.....	6
3.2 Schéma de l'enchaînement des maquettes de l'application	9
3.3 Captures d'écran d'interfaces utilisateur, adaptation web et adaptation web mobile	10
3.4 Extraits de code d'interfaces utilisateur statiques.....	12
3.5 Extraits de code de la partie dynamique d'interfaces utilisateur	13
IV. Réalisations comportant les éléments les plus significatifs côté back-end, les arguments ayant conduit à ses choix, y compris pour la sécurité	15
4.1 Présentation de la base de données (schéma conceptuel avec données et relations, schéma physique, script de création ou de modification de la base de données)	15
4.2 Extraits de code de composants métier	18
4.3 Extraits de code de composants d'accès aux données	20
V. Présentation d'éléments de sécurité de l'application	22

VI. Présentation d'un jeu d'essai élaboré par le candidat de la fonctionnalité la plus représentative (données en entrée, données attendues, données obtenues) et analyse des écarts éventuels	26
VII. Description de la veille effectuée durant le projet sur les vulnérabilités de sécurité, description des vulnérabilités éventuellement trouvées et des failles potentiellement corrigées	29
VIII. Déploiement.....	33
8.1 Environnements	33
8.2 Plan de déploiement.....	34
8.3 Plan de déploiement simplifié en local	35
IX. Références	36
X. Conclusion	37
XI. Annexes comportant les éléments de la fonctionnalité la plus représentative, côté front-end et back-end.....	38
11.1 Maquettes des interfaces utilisateur	38
11.2 Captures d'écran des interfaces utilisateur et code correspondant	39
11.3 Schéma d'architecture du modèle MVC.....	48
11.4 Code des composants métier les plus significatifs	49
11.5 Code des composants d'accès aux données les plus significatifs	56
11.6 Tableau des tests fonctionnels manuels	58
11.7 Liens vers GitHub et le site déployé sur OVH.....	60

I. Introduction avec la liste des compétences du référentiel couvertes par le projet

1.1 Développer la partie front-end d'une application web ou web mobile sécurisée

→ Installer et configurer son environnement de travail en fonction du projet web ou web mobile

Dans le cadre de ce projet, j'ai mis en place un environnement de développement adapté pour développer, tester et sécuriser l'application en local avant le déploiement sur un hébergement distant.

IDE : Visual Studio Code

Serveur local : Wamp Server

Versioning : Git avec un dépôt GitHub

Navigateur : Chrome avec outils intégrés

Matériel : Ordinateur personnel

→ Maquetter des interfaces utilisateur web ou web mobile

Avant le développement, j'ai réalisé des **zoning et wireframes sur Figma** afin de structurer l'ergonomie et l'expérience utilisateur du site. Les maquettes ont permis de définir surtout la hiérarchie des informations et la navigation entre les pages grâce au **prototype**. Elles ont servi de référence pour le développement front-end.

→ Réaliser des interfaces utilisateur statiques web ou web mobile

Les interfaces statiques ont été développées en respectant les bonnes pratiques du web. Elles constituent la base visuelle du site et garantissent une bonne expérience utilisateur.

HTML : balises sémantiques, meta description, formulaires structurés.

CSS : responsive design mobile-tablette-desktop, tailles de police lisibles.

JavaScript : Bouton retour en haut de page, animations d'apparition en fondu, AJAX avec fetch.

Accessibilité : labels ARIA, attributs alt sur les images, navigation clavier, respect des contrastes.

→ Développer la partie dynamique des interfaces utilisateur web ou web mobile

La partie dynamique du front-end repose principalement sur **JavaScript** (tri des classements, affichage des classements par saison) et **l'interaction avec le back-end PHP** (formulaires interactifs). Les données affichées côté front sont toujours **contrôlées et sécurisées** côté serveur afin d'éviter toute manipulation côté client.

1.2 Développer la partie back-end d'une application web ou web mobile sécurisée

→ Mettre en place une base de données relationnelle

Ce projet a nécessité une base de données relationnelle pour assurer une gestion fiable et cohérente des données. La modélisation a été menée avec le logiciel **Looping**, à travers la réalisation des schémas **MCD, MLD et MPD**, permettant de structurer les données et de générer le script SQL pour sa mise en place. Le choix s'est porté sur **MySQL**, administré via **phpMyAdmin**, pour sa stabilité et son adéquation aux applications web. Ce travail de conception a permis d'identifier et de hiérarchiser les clés primaires et étrangères, ainsi que de définir précisément les cardinalités entre les entités.

→ Développer des composants d'accès aux données SQL et NoSQL

Les composants d'accès aux données sont exclusivement développés en **SQL** et intégrés dans une architecture **MVC** en **PHP**. Les modèles gèrent l'ensemble des interactions avec la base MySQL : Des **requêtes SQL** permettent de calculer dynamiquement les classements et les opérations CRUD sont centralisées et sécurisées à l'aide de **requêtes préparées**, garantissant la sécurité et la fiabilité des traitements côté serveur.

→ Développer des composants métier côté serveur

Les composants métier côté serveur sont principalement implémentés dans les **Controllers**, qui orchestrent les échanges entre **Models** et **Views** dans l'architecture **MVC**. Ils gèrent la logique de l'application : actions CRUD, calcul des classements, contrôle des rôles et aspects de sécurité. Les **Controllers** valident les données et appliquent les règles métier avant toute interaction avec la base, garantissant ainsi une application cohérente, maintenable et sécurisée.

→ Documenter le déploiement d'une application dynamique

La documentation du **déploiement** décrit l'installation et la configuration de l'application, l'import de la base **MySQL** et la mise en ligne sur un **hébergeur distant**. Elle inclut un **README** garantissant une mise en production fiable et une prise en main rapide de l'application.

II. Contexte du projet

2.1 Présentation personnelle et de la communauté

Passionné d'informatique depuis mon enfance, je me suis toujours intéressé aux montages photo, vidéo, création de logos et gestion de forums, mais je ne m'étais jamais lancé dans le développement web. Cette formation s'est imposée naturellement pour compléter mes compétences et transformer ma passion en véritable **projet de reconversion professionnelle**. Ancien boulanger pendant trois ans, puis commercial pendant onze ans, je mets aujourd'hui mon expérience au service de projets personnels et communautaires. Mon objectif est désormais de faire du développement web mon **métier** et de continuer à évoluer dans ce domaine.

Depuis **2008**, cette communauté organise un championnat de courses en ligne sur PlayStation, qui en est aujourd'hui à sa 26ème saison. Le **championnat**, divisé en deux catégories, F1 et F2, regroupe des **passionnés** équipés de playseat, volants et casques micro pour communiquer pendant les courses, qui se déroulent deux fois par semaine en soirée, de 21h à 23h.

Depuis **2016**, je **participe** activement au championnat et j'en suis devenu l'un des **organiseurs**. Porté par cette communauté, ma **passion** pour la F1 s'est renforcée et m'a naturellement conduit, en **2024**, à m'engager comme **commissaire de piste** bénévole au circuit Paul Ricard du Castellet.

2.2 Cahier des charges et expression des besoins

Le projet répond à un **véritable besoin** organisationnel : jusqu'à présent, les **classements** du championnat étaient gérés **manuellement**, à partir de fichiers Excel et **d'images statiques**.

Ce processus est long, répétitif et sujet à des erreurs de calcul, rendant la gestion des saisons **fastidieuse** et la consultation des résultats **peu lisible**. La création d'une base de données dynamique et d'un site web dédié permettrait **d'automatiser** ces calculs, de centraliser les informations et d'offrir un accès **clair et fiable** aux classements, palmarès et historiques pour tous les utilisateurs.

Cette **application web** prévoit :

- Une page **Accueil** présentant le championnat avec des liens vers YouTube, Twitch, Discord et autres contenus pour le SEO.
- Une rubrique **Classements** / Palmarès / Historique dynamique, permettant filtres et tris par catégorie, saison et statistiques globales.
- Un **Dashboard sécurisé** pour gérer notamment les saisons, pilotes, équipes, Grands Prix et résultats, avec **CRUD complet** et calcul automatique des points et pénalités.
- Une page **Mentions légales et Politique de confidentialité** garantissant la transparence sur la collecte et l'utilisation des données personnelles, conformément à la réglementation en vigueur.

Les **besoins fonctionnels** incluent également la gestion des rôles (administrateur, modérateur, utilisateur), la **sécurisation** des formulaires et des sessions, ainsi que le respect du **RGPD** et de **l'accessibilité**. Sur le plan technique, le projet repose sur **PHP** en architecture **MVC**, **MySQL** pour la base de données, et des technologies front-end **HTML**, **CSS** et **JavaScript** (*voir annexe p.48 du schéma*), avec versioning **Git/GitHub**.

2.3 Contraintes du projet et livrables attendus

Pour garantir la qualité du site web, plusieurs **contraintes** sont à prendre en compte :

- **Technique** : architecture MVC, sécurisation contre les injections SQL et XSS, tokens CSRF, HTTPS obligatoire, base de données MySQL cohérente.
- **Fonctionnelle** : automatisation des calculs de classement, interface CRUD sécurisée, affichage correct des résultats et filtres pour les classements.
- **Design et accessibilité** : design responsive mobile-first, contraste et lisibilité optimisés, navigation au clavier facilitée grâce aux balises sémantiques et aux attributs ARIA pour les éléments dynamiques.

Les **livrables attendus** comprennent le code source complet, la base de données exportée (.sql), un README détaillé avec présentation de l'application et procédure de déploiement documentée.

2.4 Environnement humain et technique

L'**environnement humain** du projet comprend une petite communauté de 9 personnes, dont 3 organisateurs et 6 modérateurs, qui interagissent directement avec les pilotes pour la saisie et la validation des résultats. Bien que je sois le **seul à réaliser ce projet**, je prends en compte les besoins et les retours de cette **équipe** pour concevoir et développer l'application.

Sur le plan **technique**, le projet s'appuie sur :

- **IDE** : Visual Studio Code, avec extensions pour développer.
- **Développement local** : Wamp Server avec environnement Apache / PHP / MySQL.
- **Versioning** : Git avec un dépôt GitHub public permettant suivi des versions et historique des commits *(voir annexe p.60 du Lien QR Code vers GitHub)*.
- **Hébergements distants** : OVH pour les tests *(voir annexe p.60 du Lien QR Code vers le site)* et O2SWITCH pour le déploiement final.
- **Base de données** : MySQL administrée via phpMyAdmin.
- **Technologies front-end et back-end** : HTML, CSS, JavaScript et PHP (MVC), avec sécurité renforcée, responsive design et SEO optimisé.
- **Navigateur** : tests réalisés sur Chrome avec les outils intégrés (inspecteur, console).
- **Matériel** : Ordinateur personnel connecté à Internet, Double écran.

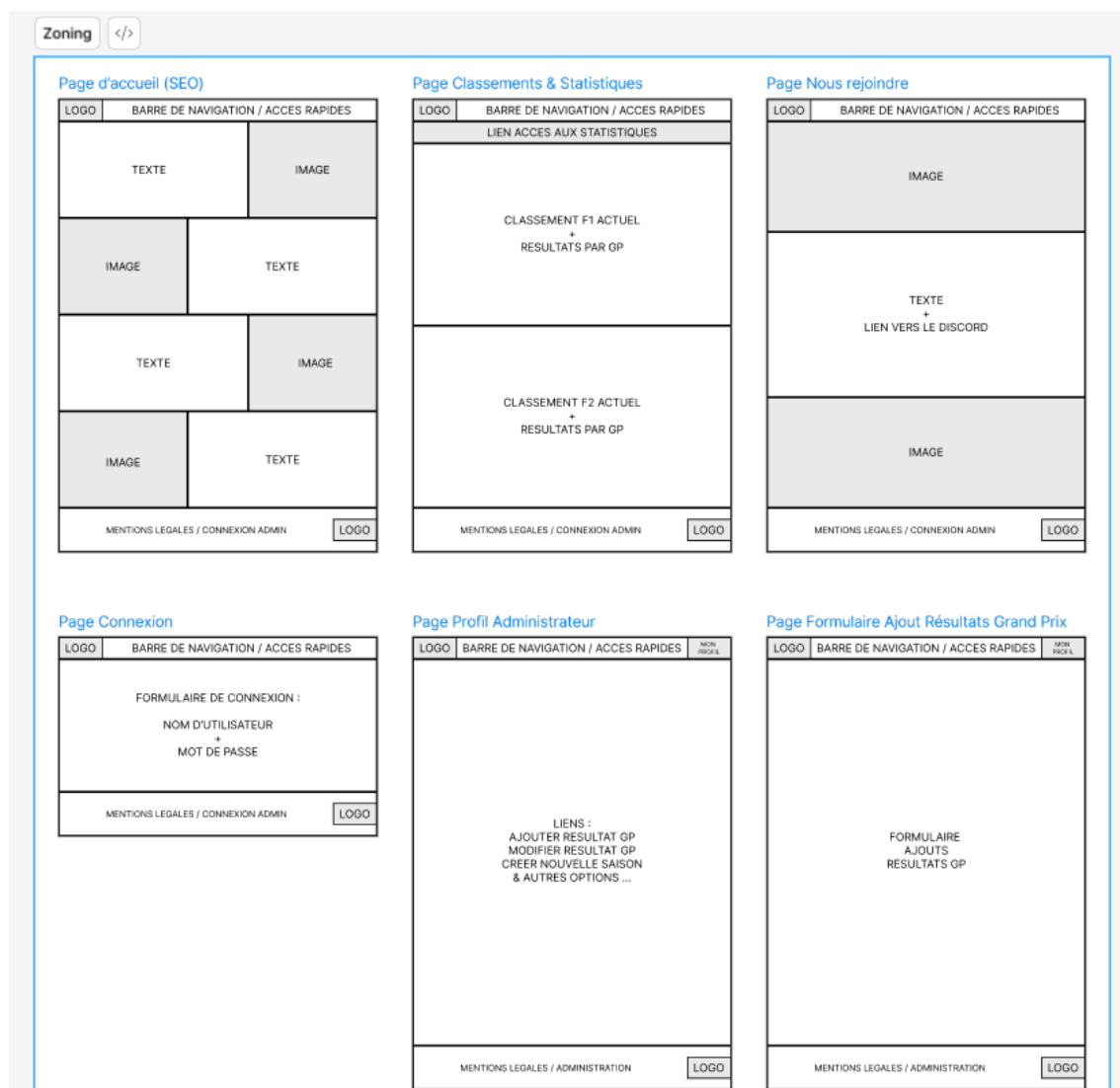
Cet **environnement humain et technique**, combinant une petite communauté active et des outils professionnels adaptés, me permet de concevoir, développer et déployer l'application web de manière **autonome** tout en garantissant **qualité, sécurité et fiabilité**.

III. Réalisations comportant les éléments les plus significatifs côté front-end, les arguments ayant conduit à ces choix, y compris pour la sécurité

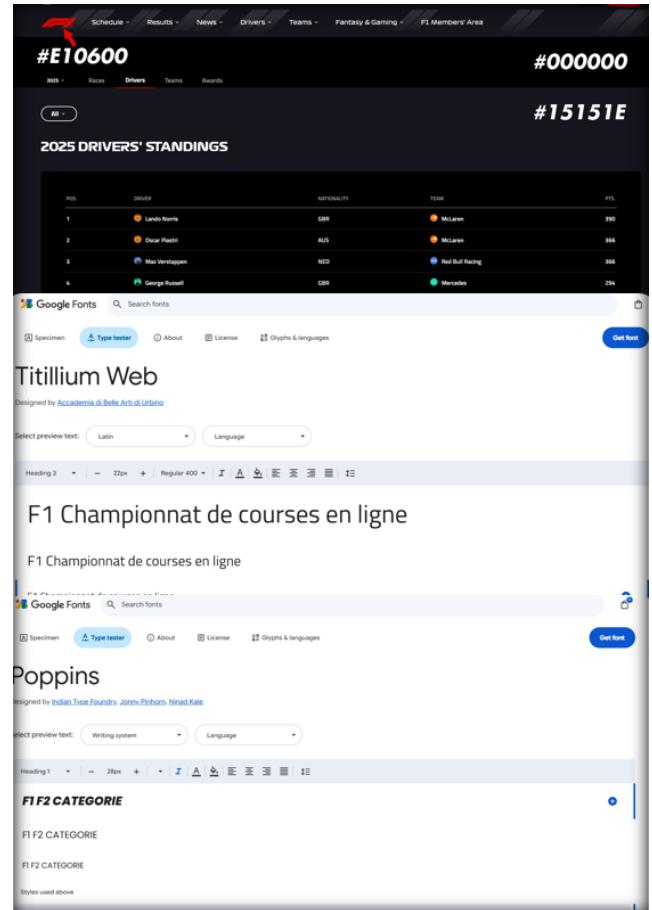
3.1 Présentation de maquettes de l'application, adaptation web et adaptation web mobile

J'ai réalisé le **zoning** sur **Figma** pour l'ensemble des **six pages** de l'application, afin de définir clairement la structure et la répartition des blocs de contenu. Il m'a permis d'organiser de manière logique les éléments principaux (navigation, contenus, actions utilisateur) sur chaque page et **d'optimiser l'UX/UI** grâce à une hiérarchisation claire de l'information. Cette phase était indispensable pour **structurer mes idées** avant la conception des wireframes et servir de base solide pour la suite du design.

Voir annexe p.38 : Lien QR Code Figma pour accéder au Zoning



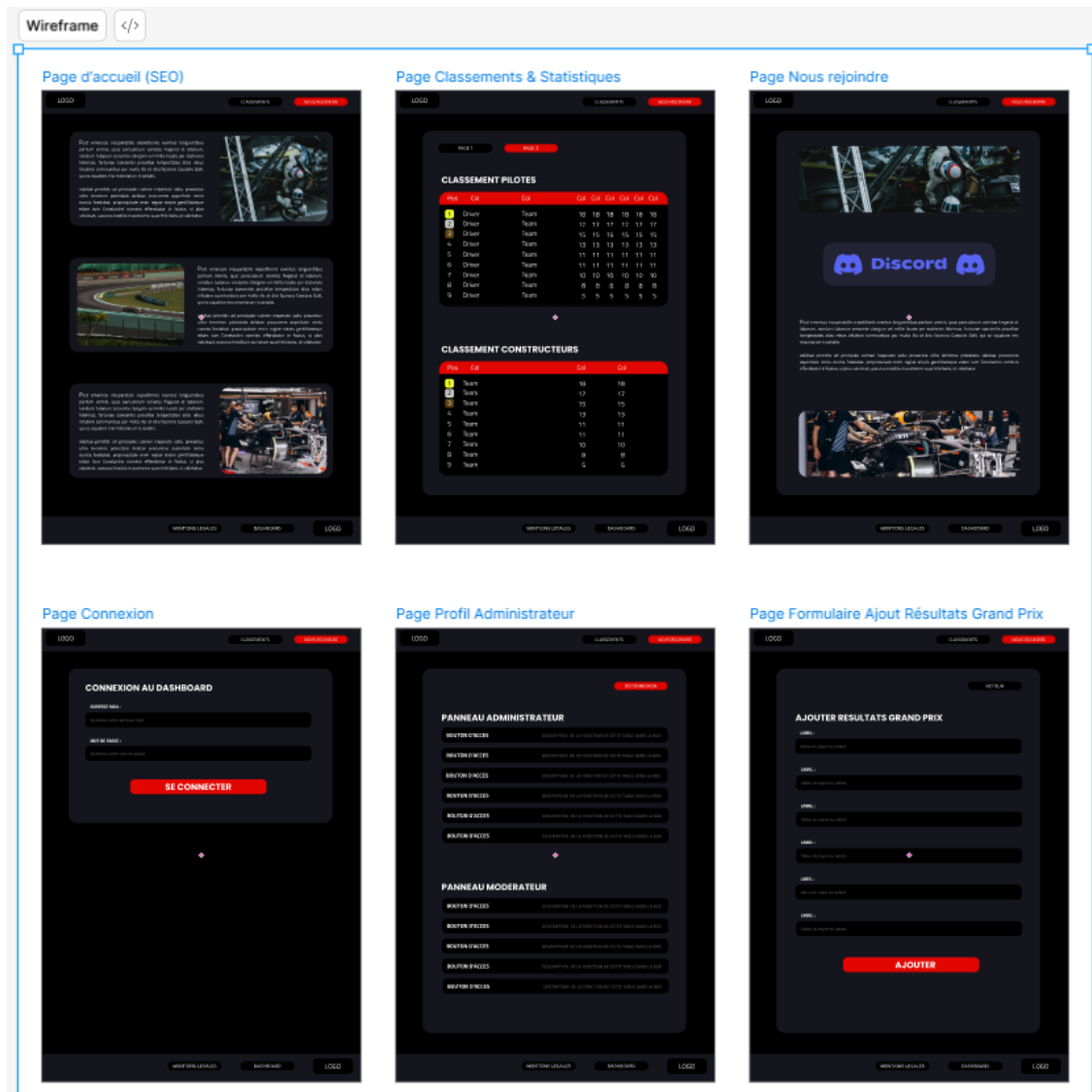
Avant de concevoir les wireframes, une étape essentielle a consisté à définir l'identité visuelle de l'application, notamment le choix des couleurs et des polices. Le design s'inspire du **site officiel de la Formule 1**, en reprenant des codes couleurs similaires et des polices libres de droit proches de celles utilisées. Trois couleurs principales ont été sélectionnées afin d'assurer une harmonie graphique optimale : le noir **#000000** pour **l'arrière-plan**, un noir éclairci **#15151E** pour **les sections**, et le rouge **#E10600** pour **les boutons et en-têtes de tableaux**. Deux polices libres de droits issues de Google Fonts ont été retenues : **Titillium Web** pour le contenu et **POPPINS** pour les titres. Cette combinaison garantit une bonne lisibilité et une distinction claire entre les différents niveaux de texte.



Une fois la charte graphique du site définie, l'étape suivante a été la conception des **wireframes**. C'est à ce stade que j'ai choisi d'intégrer les **formes arrondies** (border-radius) aux header, footer et sections, afin d'offrir un rendu plus confortable et agréable à l'œil. Cette phase m'a également permis de réfléchir à **l'optimisation de la navigation du site** en évitant la surcharge du header avec trop de boutons. Le choix d'un dashboard volontairement **simplifié** a été privilégié, dans le but de proposer une expérience utilisateur (**UX**) la plus **claire et intuitive** possible.

Enfin, la réalisation d'un **prototype** navigable sur **Figma** a constitué un atout majeur, car elle m'a permis de **valider** définitivement la **pertinence** des **choix UX/UI** et de confirmer la bonne conception globale de l'application web, avant d'attaquer la moindre ligne de code.

Voir annexe p.38 : Lien QR Code Figma pour accéder aux Wireframes



Voir annexe p.38 : Lien QR Code Figma pour accéder au Prototype



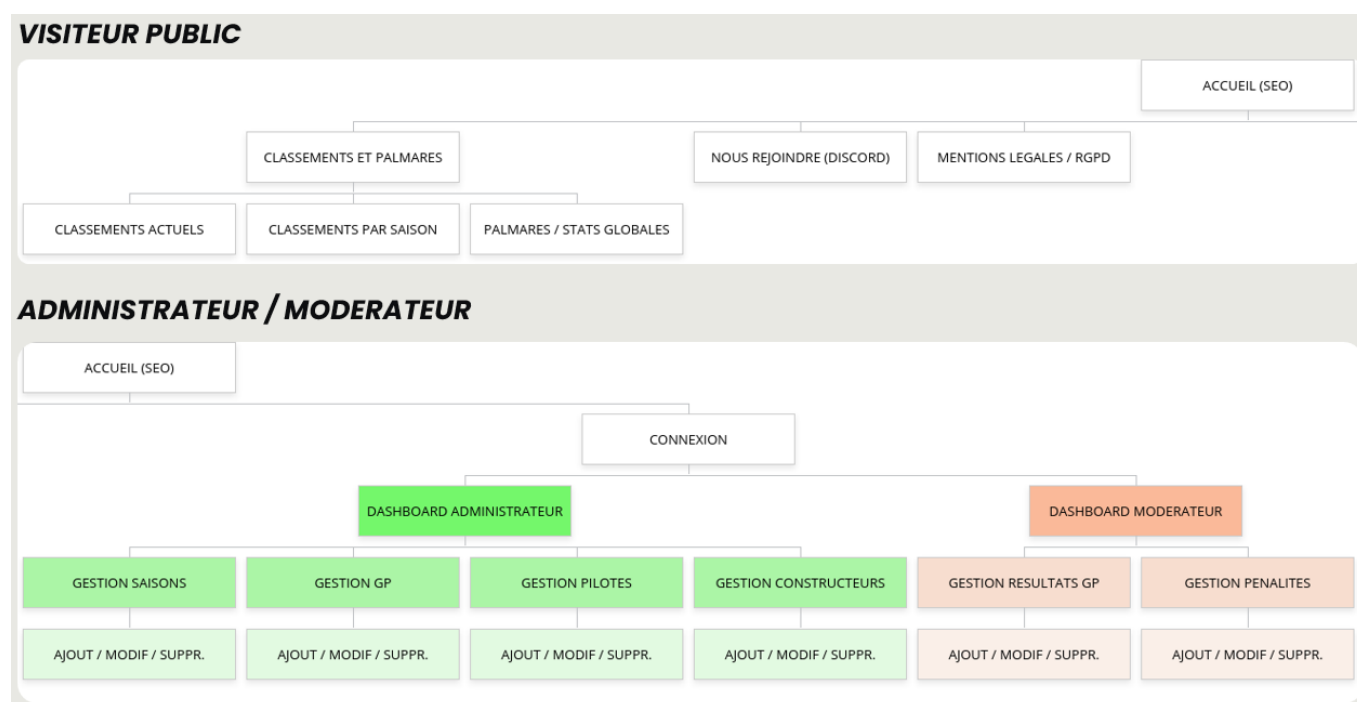
3.2 Schéma de l'enchaînement des maquettes de l'application

Le schéma d'enchaînement des maquettes de l'application, réalisé avec **GlooMaps.com**, représente l'organisation des pages et les différents parcours utilisateurs. Il distingue **les parcours** des visiteurs publics et ceux des administrateurs/modérateurs.

Le **visiteur** accède d'abord à la page d'accueil, qui présente les informations essentielles pour découvrir la communauté. Il peut ensuite naviguer vers les sections « Classements » et « Palmarès », afin de consulter les classements actuels, les classements par saison ainsi que les statistiques globales. Les pages d'informations complémentaires (Discord, Mentions légales) restent accessibles depuis toutes les pages.

Pour les **administrateurs et modérateurs**, un accès dédié au tableau de bord est disponible via un bouton « Dashboard » présent dans le footer. Cet espace sécurisé permet d'accéder à la gestion du championnat, avec des droits accordés selon le rôle.

Ce schéma met en évidence les liens entre les pages publiques et les espaces sécurisés, et montre comment la structure de l'application garantit une navigation fluide et adaptée à chaque type d'utilisateur.



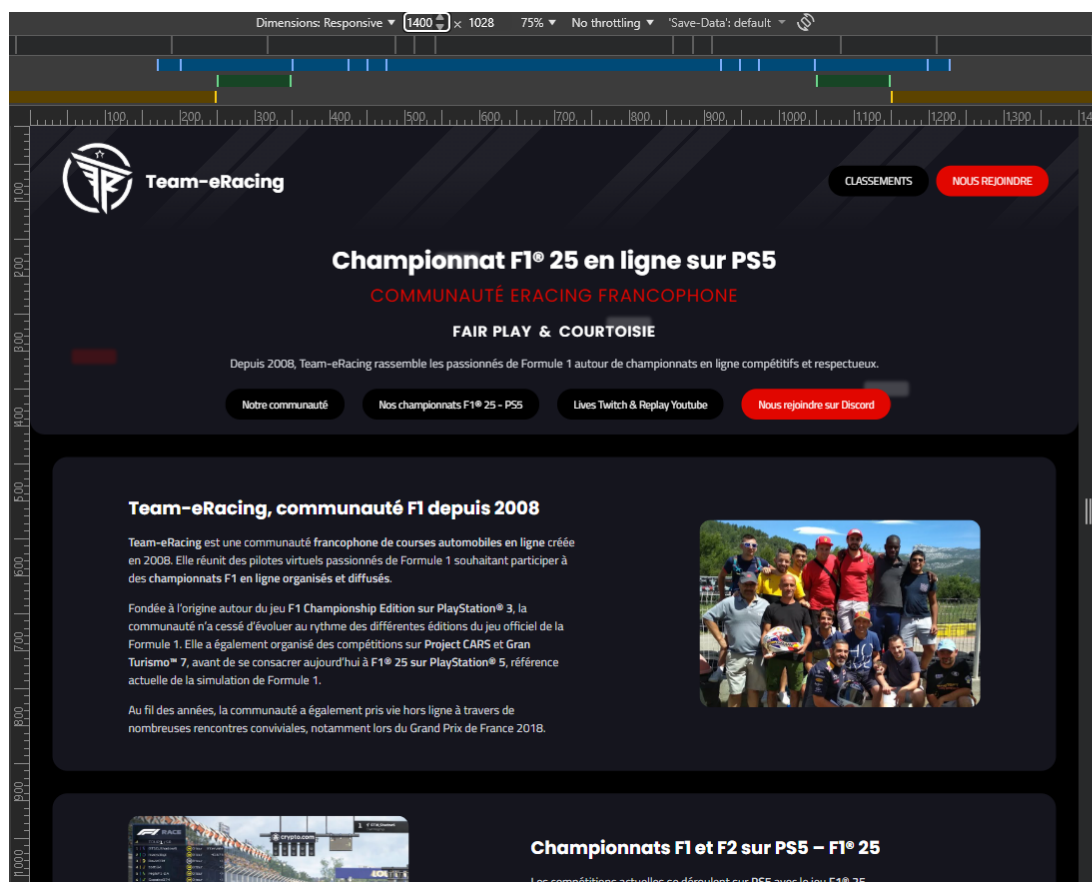
3.3 Captures d'écran d'interfaces utilisateur, adaptation web et adaptation web mobile

La **page d'accueil** présente le championnat, la communauté et les principaux **boutons d'action** dès l'arrivée sur le site : accès aux classements, nous rejoindre (en rouge) et navigation vers les différentes sections de la page. Le contenu est organisé en blocs avec un titre principal, un sous-titre et une zone de présentation de la communauté accompagnée d'une image.

Sous le header, une section de présentation succincte de la communauté intègre une **animation CSS** discrète et moderne, évoquant le passage des monoplaces de Formule 1 et renforçant **l'identité visuelle du site**.

L'interface est réalisée en **responsive design** : sur **desktop**, la mise en page utilise des blocs disposés en largeur, avec textes et images juxtaposés. Sur **mobile**, le contenu est réorganisé en colonne unique, les images se redimensionnent automatiquement et les textes s'adaptent à la largeur de l'écran.

DESKTOP



MOBILE



La **page Classements** permet aux utilisateurs de consulter les résultats des championnats. Elle affiche les classements pilotes, constructeurs et la liste des GP de la saison sous forme de **grands tableaux**.

Un **bandeau d'en-tête** rappelle la saison sélectionnée ainsi que la dernière date de mise à jour, ce qui aide l'utilisateur à **comprendre** immédiatement le **contexte** des données affichées. La hiérarchie visuelle met en avant la catégorie, puis le détail de la saison choisie.

La mise en place du **responsive design** sur ces pages a représenté la **principale difficulté**. Les **tableaux** sont naturellement très **larges** en version **desktop** et comportent de **nombreuses colonnes**. Sur **mobile**, ils ont été repensés pour rester **complets** et lisibles **sans masquer** d'informations essentielles : abréviations pour les en-têtes des tableaux, cellules compactées, tailles de police ajustées et optimisation des espacements.

Ce **principe** de responsive design a été appliqué de façon similaire à l'ensemble des **tableaux** du **Dashboard**. Ainsi, administrateurs et modérateurs peuvent consulter, modifier et gérer les données facilement, même depuis un **smartphone**.

DESKTOP

Dimensions: Responsive 1400 x 1028 75% No throttling 'Save-Data': default

 **Team-eRacing**

CLASSEMENTS NOUS REJOINDRE

Palmarès Circuits

Classements
Dernière mise à jour : 31/12/2025 14:24 / F2 - Saison 1 - GP 3 - Melbourne (Australie)

Saison 1 - F1 Championship Edition - PS3

F1 Saison 1 F1 Championship Edition - PS3

Classement Pilotes F1

Pilote	Équipe	Pts	GP	Vict	Podi	Pole	FastestL.
1 Jujubiker		141	16	8	15	0	0
2 Martio		68	7	6	7	0	0
3 Guignol81		57	17	0	5	0	0
4 Jimboparisgo		51	14	0	4	0	0
5 Chapi-chapo		38	9	0	2	0	0
6 Didi511		29	9	0	1	0	0
7 Senna76		23	7	0	1	0	0
8 Fox		13	4	0	1	0	0
9 Nordschleife		11	5	0	1	0	0
10 Ludovico6		5	4	0	0	0	0
11 Gafit		4	9	0	0	0	0
12 [Driver removed]		0	12	4	17	0	0

Classement Constructeurs F1

MOBILE

400 x 1028 75% No throttling 'Save-Data': default

 **Team-eRacing**

CLASSEMENTS NOUS REJOINDRE

Palmarès Circuits

Classements
MAJ : 31/12/2025 14:24

Saison 1 - F1 Championship Edition - F

F1 Saison 1 F1 Championship Edition - PS3

Classement Pilotes F1

Pilote	Equ	Pts	GP	Vi	Pa	Po	FL
1 Jujubiker		141	16	8	15	0	0
2 Martio		68	7	6	7	0	0
3 Guignol81		57	17	0	5	0	0
4 Jimboparisgo		51	14	0	4	0	0
5 Chapi-chapo		38	9	0	2	0	0
6 Didi511		29	9	0	1	0	0
7 Senna76		23	7	0	1	0	0
8 Fox		13	4	0	1	0	0
9 Nordschl		11	5	0	1	0	0
10 Ludovico		5	4	0	0	0	0
11 Gafit		4	9	0	0	0	0
12 [Driver		0	12	4	17	0	0

Classement Constructeurs F1

Équipe	Points
 Toyota	141

3.4 Extraits de code d'interfaces utilisateur statiques

Ces extraits illustrent la mise en place des **interfaces statiques** du site.

Dans le fichier **base.php**, la structure **HTML** respecte les bonnes pratiques : déclaration du **doctype**, **balises sémantiques**, **métadonnées** pour le responsive (**meta viewport**), **favicon** et **feuilles de styles** organisées. La page est structurée autour des **balises** header, nav, main et footer, ce qui améliore l'**accessibilité**, la compréhension du code et le **référencement**. Cette structure de base est utilisée sur l'ensemble des pages. Grâce au modèle MVC, seul le contenu de la balise **<main>** est mis à jour en fonction des vues appelées via la variable **\$content**.

```
base.php
You, yesterday | 1 author (You)
<!DOCTYPE html>
<html lang="fr">

<head>
<meta charset="UTF-8">
<meta name="viewport">
<title><?= htmlspecialchars($title) >
<!-- Favicon classique -->
<link rel="icon" type="image/x-icon" href="img/favicon.ico">
<!-- iPhone / iPad échelle 1x -->
<link rel="apple-touch-icon" href="img/icon-180.png">
<!-- Android échelle 1x -->
<link rel="manifest" href="manifest.json">
<!-- Couleur de thème -->
<meta name="theme-color" content="#000000">
<meta name="description">
<meta name="robots" content="index, follow">
<link rel="canonical" href="https://www.team-e-racing.com">
<!-- Google Fonts pour le titre -->
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=block" rel="stylesheet">
<!-- CSS personnel, pour le contenu -->
<link rel="stylesheet" href="css/main.css">
<link rel="stylesheet" href="css/nav.css">
<link rel="stylesheet" href="css/footer.css">
<!-- Icones Vectorielles -->

<body>
<header>
<div class="logo_and_name_header">
<div class="logo_header">
<a href="index.php">
</div>
<span class="name_header">
Team-eRacing
</span>
</div>
<nav>
<a class="nav-btn" href="index.php">Accueil</a>
<a class="nav-btn red" href="about.php">À propos</a>
</nav>
</header>

<main>
<!-- Affichage dynamique de la vue -->
<?= $content ?>
</main>

<footer>
<nav aria-label="Liens">
<a class="nav-btn" href="index.php">Accueil</a>
<a class="nav-btn" href="about.php">À propos</a>
</nav>
<div class="logo_footer">
</div>
</footer>
```

Sur la page d'accueil **index.php**, les sections "hero" utilisent des **balises structurantes** (section, div, h2, p) pour améliorer l'**accessibilité** et la **lisibilité** du code. Le contenu est hiérarchisé afin de mettre en avant les informations principales. Les autres sections du site sont construites selon les mêmes conventions. Des **balises sémantiques** **** sont également utilisées pour mettre en avant les informations clés et améliorer la compréhension du contenu par les **utilisateurs** et les **moteurs de recherche**.

```
Views > home > index.php
1 <section class="hero">
10 <div class="hero-content">
27 <div class="hero-cta">
32 </div>
33 </div>
34 </section>
35
36
37 <section class="hero-section flex-reverse" id="presentation">
38 <div class="hero-text-block">
39 <h2>Team-eRacing, communauté F1 depuis 2008</h2>
40 <p>
41 <strong>Team-eRacing</strong> est une communauté créée en 2008. Elle réunit des pilotes virtuels participants à des <strong>championnats F1 en ligne</strong>
42 </p>
43 <p>
44 Fondée à l'origine autour du jeu <strong>F1 Championship Manager</strong>, la communauté n'a cessé d'évoluer au rythme des développements. Elle a également organisé des compétitions sur <strong>F1® 25 sur PlayStation® 5</strong> référencées sur le site.
45 </p>
46 <p>
47 Au fil des années, la communauté a également pris
48 </p>
49 </div>
50 
51
52
53
54
55
56
57
58
```

Enfin, les extraits **CSS** présentés mettent en évidence une **organisation** claire des feuilles de styles orientée **responsive design**. J'utilise **plusieurs fichiers CSS** selon les largeurs d'écran, avec des feuilles spécifiques pour les résolutions inférieures à 700 px, 900 px et 1400 px. Cette organisation me permet une meilleure **lisibilité** et **maintenabilité** du code.

En complément, j'utilise des **media queries** pour ajuster finement certains composants en fonction de la taille de l'écran. Ce principe est appliqué de manière cohérente à l'ensemble des pages du site, garantissant une expérience utilisateur (**UX**) et une interface optimale (**UI**) aussi bien sur **mobile** que sur **desktop** (voir annexe p.39 des captures d'écran et extraits de code).

```
public > # style700px-mobilev1.7.css > {} @media screen
39 .logo_footer p {
42 }
43
44 /* Page Accueil / Index */
45 .hero h1 {
46     font-size: 1.5rem;
47 }
48
49 .community-tag {
50     font-size: 1.25rem;
51 }
52
53 /* Styles pour écrans < 450px */
54 @media screen and (max-width: 450px) {
55     .hero h1 {
56         font-size: 1.3rem;
57     }
58
59     .community-tag {
60         font-size: 1.1rem;
61     }
62 }
```

3.5 Extraits de code de la partie dynamique d'interfaces utilisateur

Ces extraits illustrent la **dynamique des interfaces** utilisateur, depuis la récupération des données via les modèles, leur traitement dans les contrôleurs, jusqu'à leur affichage sécurisé dans les vues.

Views/dashboard/circuits/index.php

Cette partie de code montre l'affichage **dynamique** de la liste des circuits dans une vue **MVC**. Les données sont transmises par le contrôleur sous forme de **tableau d'objets** et parcourues à l'aide d'une boucle **foreach**. Les valeurs affichées sont systématiquement protégées par la fonction **htmlspecialchars**, afin d'éviter les failles de type XSS lors de l'affichage des données issues de la base.

```
ss="table-responsive">
le class="dashboard-table fix">


```

Ces deux exemples illustrent la gestion des interfaces utilisateur au sein de l'architecture MVC.

Le contrôleur **CircuitsController** joue un rôle central en récupérant les données nécessaires à l'affichage via le modèle **CircuitsModel**. Celui-ci est responsable de l'accès à la base de données et regroupe les requêtes SQL, notamment pour récupérer la liste des circuits et des pays associés. Les données obtenues sont ensuite transmises à la **vue** à l'aide de la méthode **render**, héritée du contrôleur principal, sous forme de variables exploitables dans l'interface utilisateur.

Cette séparation claire entre logique métier, accès aux données et affichage sécurisé permet de générer **dynamiquement** le contenu des pages.

```
class CircuitsController extends Controller {  
  
    // INDEX : liste des circuits  
    public function index()  
    {  
        $circuits = CircuitsModel::all();  
        $countries = CircuitsModel::allCountries();  
  
        $this->render('dashboard/circuits/index', [  
            'list' => $circuits,  
            'countries' => $countries  
        ]);  
    }  
}
```

```
class CircuitsModel extends DbConnect {  
  
    // Récupérer tous les circuits  
    public static function all(){  
        $db = new DbConnect();  
        return $db->getConnection()->query("  
            SELECT circuits.id, circuits.name, circ  
            FROM circuits  
            JOIN countries ON circuits.country_id =  
            ")->fetchAll();  
    }  
  
    // Récupérer tous les pays  
    public static function allCountries(){  
        $db = new DbConnect();  
        return $db->getConnection()->query("  
            SELECT id, name  
            FROM countries  
            ORDER BY name ASC  
            ")->fetchAll();  
    }  
}
```

Views/dashboard/classements/standings.php

L'extrait présenté ici a permis d'améliorer **l'interactivité** de l'interface tout en évitant la **surcharge** de la page **Classements**. L'affichage des résultats détaillés des Grands Prix est réalisé via une requête **AJAX** utilisant **fetch**. Lorsqu'un utilisateur clique sur un GP, une requête **asynchrone** est envoyée au contrôleur **ClassementsController** pour récupérer uniquement les détails du GP sélectionné. Cette approche permet de charger le contenu dans un **modal** sans recharger toute la page. Le choix de **fetch** s'explique par sa **simplicité** et sa **compatibilité** avec les navigateurs modernes.

```
<!-- Résultats GP -->  
<tr class="gp-row" data-gp-id="<?= (int)$gp->id ?>">  
  
</tr>  
  
<script>  
/* Ouvre le modal Résultats du GP en cliquant sur une ligne GP */  
document.addEventListener('click', function (e) {  
    const row = e.target.closest('.gp-row');  
    if (!row) return;  
  
    const gpId = row.dataset.gpId;  
    if (!gpId) return;  
  
    fetch(`index.php?controller=classements&action=gpDetails&gp_id=${gpId}`)  
        .then(res => res.text())  
        .then(html => {  
            document.getElementById('gp-modal-body').innerHTML = html;  
            document.getElementById('gp-modal').style.display = 'block';  
            updateResponsiveNames();  
        });  
});  
  
/* Fermer le modal */  
document.querySelector('.gp-modal-close').addEventListener('click', () => {  
    document.getElementById('gp-modal').style.display = 'none';  
});  
</script>
```

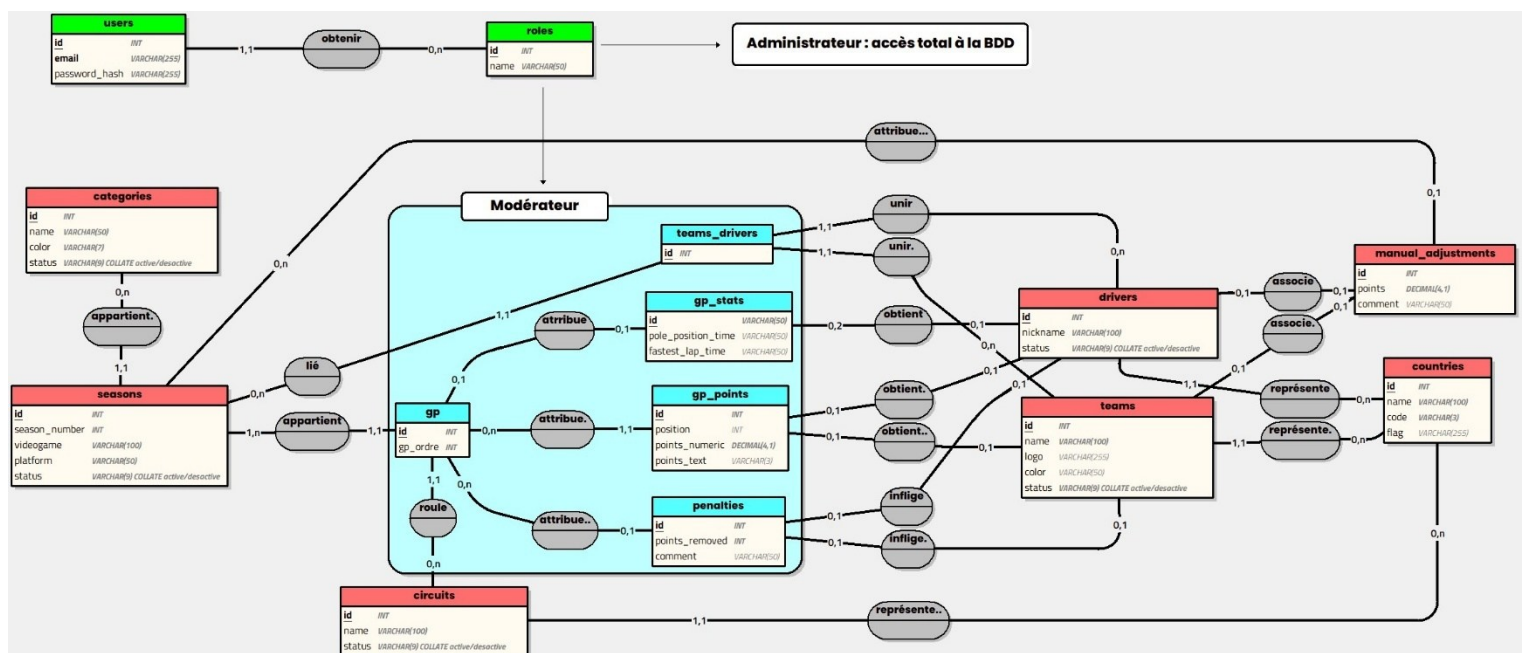
IV. Réalisations comportant les éléments les plus significatifs côté back-end, les arguments ayant conduit à ses choix, y compris pour la sécurité

4.1 Présentation de la base de données (schéma conceptuel avec données et relations, schéma physique, script de création ou de modification de la base de données)

Ce **modèle conceptuel de données (MCD)**, réalisé avec le logiciel **Looping** selon la **méthode Merise**, présente la structure globale de la base de données et les relations entre les différentes entités, telles que les utilisateurs, les rôles, les catégories, les saisons, les circuits, les Grands Prix, les points, les statistiques, les équipes et les pilotes.

J'ai conçu ce schéma comme une vue d'ensemble afin de rendre plus lisible le fonctionnement des données, notamment pour la gestion des résultats, des points, des pénalités et des droits d'accès. Cette étape était cruciale au vu du nombre important de **tables** composant la **base de données**.

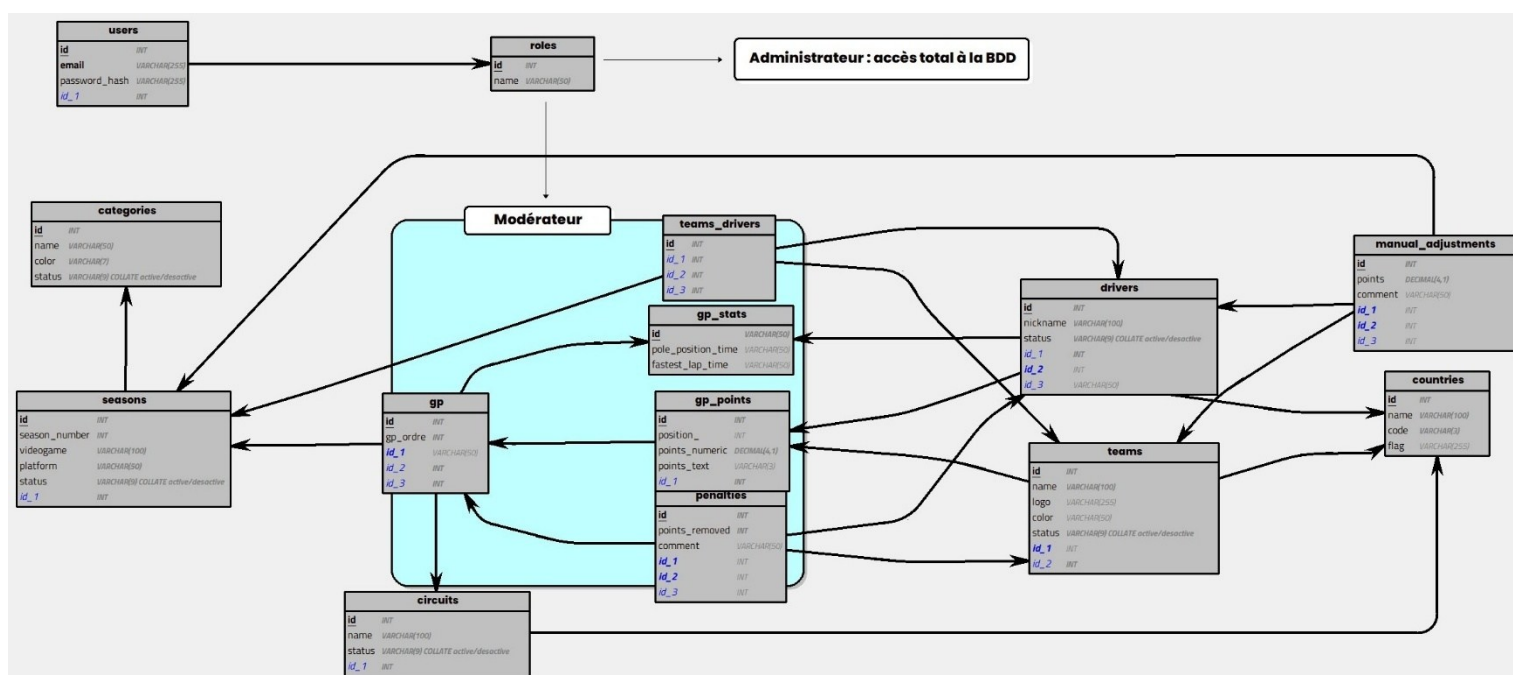
L'un des principaux défis a été de définir correctement les **cardinalités** entre les **entités**, afin de respecter les règles métier et d'éviter des incohérences. Ce travail a permis de mettre au clair le **fonctionnement conceptuel** de l'application.



Le logiciel **Looping** permet de générer le **Modèle Logique de Données (MLD)** tout en vérifiant la cohérence des relations définies dans le **MCD**.

Chaque entité est traduite en **table relationnelle** avec une **clé primaire**, et les relations sont matérialisées par des **clés étrangères** (champs précédés de #). Par exemple, la table **users** contient une clé étrangère vers **roles**, ce qui permet d'associer chaque utilisateur à un rôle précis. De la même manière, la table **seasons** référence une **categorie**, tandis que la table **gp** est liée à une **season**, un **circuit** et des **statistiques** de **Grands Prix**. Les relations entre plusieurs entités, comme entre teams et drivers, sont gérées via des tables intermédiaires telles que **teams_drivers**.

Ce passage du MCD au MLD permet de valider les **cardinalités** et de préparer une base de données exploitable pour **l'implémentation physique**.



À partir du **MCD** validé, **Looping** permet également de créer automatiquement le **Modèle Physique de Données (MPD)** sous forme d'un **script SQL** directement exploitable. Ce script traduit concrètement le MLD en instructions **CREATE TABLE**, en définissant les types de données, les clés primaires, les contraintes d'unicité et les clés étrangères.

Par exemple, la table **users** intègre une contrainte d'unicité sur l'**email** ainsi qu'une **clé étrangère** vers la table **roles**, garantissant une gestion cohérente des droits. Les relations entre les entités sont matérialisées par des contraintes **FOREIGN KEY**, comme entre **gp** et **seasons**, **circuits** ou **gp_stats**, assurant l'intégrité référentielle.

La génération automatique du script réduit les erreurs humaines et permet de **déployer** rapidement une **base structurée** et **prête** à être utilisée côté back-end.

```
-----
MPD -----
Script SQL généré -----
-----

CREATE TABLE roles(
  id INT,
  name VARCHAR(50) NOT NULL,
  PRIMARY KEY(id)
);

CREATE TABLE categories(
  id INT,
  name VARCHAR(50) NOT NULL,
  color VARCHAR(7) NOT NULL,
  status VARCHAR(9) COLLATE active/desactive,
  PRIMARY KEY(id)
);

CREATE TABLE countries(
  id INT,
  name VARCHAR(100) NOT NULL,
  code VARCHAR(3) NOT NULL,
  flag VARCHAR(255),
  PRIMARY KEY(id)
);

CREATE TABLE gp_stats(
  id VARCHAR(50),
  pole_position_time VARCHAR(100) NOT NULL,
  fastest_lap_time VARCHAR(50) NOT NULL,
  PRIMARY KEY(id)
);

CREATE TABLE seasons(
  id INT,
  season_number INT NOT NULL,
  videogame VARCHAR(100) NOT NULL,
  platform VARCHAR(50) NOT NULL,
  status VARCHAR(9) COLLATE active/desactive,
  id_1 INT NOT NULL,
  PRIMARY KEY(id),
  FOREIGN KEY(id_1) REFERENCES seasons(id)
);

CREATE TABLE gp_points(
  id INT,
  position INT,
  points_numeric DECIMAL(4,1) NOT NULL,
  points_text VARCHAR(3),
  id_1 INT NOT NULL,
  PRIMARY KEY(id),
  FOREIGN KEY(id_1) REFERENCES gp(id)
);

CREATE TABLE users(
  id INT,
  email VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  id_1 INT NOT NULL,
  PRIMARY KEY(id),
  UNIQUE(email),
  FOREIGN KEY(id_1) REFERENCES roles(id)
);

CREATE TABLE teams(
  id INT,
  name VARCHAR(100) NOT NULL,
  logo VARCHAR(255),
  color VARCHAR(50),
  status VARCHAR(9) COLLATE active/desactive,
  id_1 INT,
  id_2 INT NOT NULL,
  PRIMARY KEY(id),
  UNIQUE(id_1),
  FOREIGN KEY(id_1) REFERENCES gp_points(id),
  FOREIGN KEY(id_2) REFERENCES countries(id)
);

CREATE TABLE teams_drivers(
  id INT,
  id_1 INT NOT NULL,
  id_2 INT NOT NULL,
  id_3 INT NOT NULL,
  PRIMARY KEY(id),
  FOREIGN KEY(id_1) REFERENCES teams(id),
  FOREIGN KEY(id_2) REFERENCES drivers(id),
  FOREIGN KEY(id_3) REFERENCES gp_stats(id)
);

CREATE TABLE drivers(
  id INT,
  nickname VARCHAR(100) NOT NULL,
  status VARCHAR(9) COLLATE active/desactive,
  id_1 INT NOT NULL,
  id_2 INT,
  id_3 VARCHAR(50),
  PRIMARY KEY(id),
  UNIQUE(id_2),
  FOREIGN KEY(id_1) REFERENCES countries(id),
  FOREIGN KEY(id_2) REFERENCES gp_points(id),
  FOREIGN KEY(id_3) REFERENCES gp_stats(id)
);

CREATE TABLE penalties(
  id INT,
  points_removed INT NOT NULL,
  comment VARCHAR(50),
  id_1 INT,
  id_2 INT,
  id_3 INT,
  PRIMARY KEY(id),
  UNIQUE(id_1),
  UNIQUE(id_2),
  FOREIGN KEY(id_1) REFERENCES drivers(id),
  FOREIGN KEY(id_2) REFERENCES teams(id),
  FOREIGN KEY(id_3) REFERENCES gp_stats(id)
);

CREATE TABLE manual_adjustments(
  id INT,
  points DECIMAL(4,1) NOT NULL,
  comment VARCHAR(50),
  id_1 INT,
  id_2 INT,
  id_3 INT,
  PRIMARY KEY(id),
  UNIQUE(id_1),
  UNIQUE(id_2),
  FOREIGN KEY(id_1) REFERENCES drivers(id),
  FOREIGN KEY(id_2) REFERENCES teams(id),
  FOREIGN KEY(id_3) REFERENCES gp_stats(id)
);
```

4.2 Extraits de code de composants métier

Le back-end de l'application repose sur une architecture MVC, dans laquelle les **composants métier** sont principalement implémentés au sein des **contrôleurs** et des **modèles**. Une attention particulière a été portée à la **sécurité**, à la **maintenabilité** et à la **réutilisabilité** du code.

Les extraits ci-dessous illustrent les éléments les plus significatifs du back-end, notamment la gestion de **l'authentification**, des **rôles**, de la protection **CSRF** et des **interactions** avec la base de données.

La **majorité** de la logique métier est située dans les **contrôleurs**. J'ai également fait le choix d'implémenter **certaines règles métier** directement liées aux données (**algorithmes** de calcul des points, classements, pénalités et ajustements manuels) au niveau des **modèles** sous forme de **requêtes SQL**. Ces éléments sont présentés dans la **section 4.3** consacrée à l'accès aux données.

→ Sécurité centralisée

Le fichier **Controller.php** définit une classe **abstraite Controller**, dont **héritent** tous les **contrôleurs** de l'application. Cette classe met en œuvre une méthode **authMiddleware**, qui s'appuie sur les mécanismes de sécurité fournis par le cœur de l'application via la classe **Auth**. Si l'utilisateur n'est pas authentifié, il est redirigé vers la page de connexion.

```
abstract class Controller
{
    // Verification à chaque appel des Controllers :
    // LOGIN + SESSION + TIMEACTIVITY 5 MIN + ROLES +
    protected function authMiddleware(string|array $roles)
    {
        \App\Core\Auth::start();

        if (!\App\Core\Auth::check()) {
            header('Location: index.php?controller=aut
            exit;
        }
    }
}
```

L'objectif de ce choix est de **centraliser** les règles de **sécurité** et de **contrôle** d'accès, afin d'éviter les duplications de code et les oublis dans chaque contrôleur métier. De cette manière, l'authentification de l'utilisateur est **systématiquement** vérifiée à chaque appel de contrôleur, sans avoir à répéter cette logique dans chaque classe.

Le contrôleur accepte un ou plusieurs rôles, ce qui permet de **restreindre facilement** l'accès aux différentes fonctionnalités selon le profil de l'utilisateur. La logique de **contrôle des droits** est ainsi regroupée, ce qui facilite la maintenabilité et l'évolution de la gestion des autorisations.

```
// Vérification des rôles
if ($requiredRoles) {
    $userHasRole = false;

    if (is_array($requiredRoles)) {
        foreach ($requiredRoles as $role) {
            if (\App\Core\Auth::hasRole($role)) {
                $userHasRole = true;
                break;
            }
        }
    } else {
        $userHasRole = \App\Core\Auth::hasRole($
    }
}
```

Une vérification du **token CSRF** est également effectuée pour toute requête de type POST. En cas d'échec, une erreur HTTP 403 est renvoyée et l'exécution est interrompue.

Cette **protection** permet de prévenir les **attaques CSRF** sur l'ensemble des actions du **CRUD**, grâce à la **centralisation** des mécanismes de sécurité.

```
// Vérification CSRF pour POST
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    if (!isset($_POST['csrf_token']))
        || !App\Core\Auth::validateCSRF($_POST['csrf_token'])
    {
        http_response_code(403);
        // Titre de la page
        $title = "Team-eRacing";

        // Contenu affichant le message d'erreur
        $content = "<div class='section-dashboard'>
                    <h1>Token invalide !</h1>
                    <p>Veuillez réessayer ou recharger la page</p>
                    <a class='nav-btn-dashboard' href='index.php'>Retour</a>
                </div>";

        include dirname(__DIR__) . '/Views/base.php';
        exit;
    }
}
```

→ Contrôleur métier : gestion des pilotes

Le **DriversController** gère les **règles métier** liées à l'affichage, la création, la mise à jour et la suppression des pilotes. Il **hérite** du contrôleur abstrait, ce qui lui permet de **bénéficier** automatiquement de l'ensemble des mécanismes de sécurité présentés précédemment.

```
class DriversController extends Controller {

    public function __construct()
    {
        // Seuls les administrateurs peuvent accéder
        $this->authMiddleware("Administrateur");
    }
}
```

Les règles de sécurité sont définies dès la **construction** de l'objet, ce qui restreint l'accès à ce contrôleur aux utilisateurs disposant du rôle Administrateur, **avant toute exécution** des méthodes du CRUD.

Ici, dans la méthode **create**, les données saisies par l'utilisateur sont validées côté serveur avant tout traitement métier grâce à la méthode **Form::validatePost**, qui vérifie la présence des champs obligatoires. Un nettoyage minimal des entrées est ensuite appliqué afin de limiter les erreurs de saisie.

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {

    if (Form::validatePost($_POST, ['nickname', 'country_id', 'status']))
    {
        // Supprime les espaces au début et à la fin de
        // pour comparer si doublon (laisse des espaces)
        $nickname = trim($_POST['nickname']);
        $country_id = $_POST['country_id'];
        $status = $_POST['status'];

        $db = new DriversModel();
        $pdo = $db->getConnection();
    }
}
```

Cette validation **sépare** clairement la **gestion des données** de la **logique métier** et améliore la fiabilité des informations enregistrées.

L'accès à la base de données est effectué via **PDO** et des **requêtes préparées**. Ce choix permet de **sécuriser** les échanges avec la base en empêchant les **injections SQL** et en isolant les données utilisateur de la requête. Cela respecte les **bonnes pratiques** PHP en matière de sécurité et d'organisation du code. J'ai aussi choisi PDO car il offre un bon **compromis** entre performance, simplicité et sécurité, tout en restant natif en PHP.

Les opérations du CRUD sont encadrées par un bloc **try/catch** afin de gérer correctement les erreurs liées à la base de données. Une gestion **spécifique** est prévue pour les violations de contrainte d'unicité, comme ici si le pseudo existe déjà, permettant de retourner un message **compréhensible** à l'utilisateur. Les autres erreurs sont traitées avec des messages **personnalisés** afin d'éviter l'affichage d'informations techniques sensibles. Cette **gestion des exceptions** permet d'améliorer la fiabilité de l'application tout en renforçant la sécurité (*voir annexe p.49 du code de composants métier*).

```
try {
    // Requete préparée
    $stmt = $pdo->prepare("
        INSERT INTO drivers (nickname, country_id,
        VALUES (?, ?, ?)
    ");

    if ($stmt->execute([$nickname, $country_id, $pseudo]) {
        $message = "Pilote créée avec succès";
        $classMsg = "msg-success";
    } else {
        $message = "Erreur lors de la création.";
        $classMsg = "msg-error";
    }
}
```

```
} catch (\PDOException $e) {
    if ($e->getCode() == 23000) {
        $message = "Création échouée : ce pseudo existe déjà";
    } else {
        $message = "Erreur lors de la création.";
    }

    $classMsg = "msg-error";
}
```

4.3 Extraits de code de composants d'accès aux données

Les extraits suivants présentent différents **composants d'accès aux données**. Ils illustrent la manière dont les méthodes sont **organisées**, comment les **requêtes SQL** sont construites et comment la **logique** liée aux données est regroupée.

→ Connexion à la base – Core/DbConnect.php

Cette classe gère la **connexion** centralisée à la base via **PDO**. Elle applique plusieurs options de sécurité via les **attributs PDO** et expose la méthode **getConnection()** afin que les modèles puissent utiliser la **connexion sans casser l'encapsulation**. En regroupant la configuration en un seul endroit, la maintenance est simplifiée et les risques d'erreurs sont réduits. Cette classe sert de **fondation** à tous les **accès aux données**.

```
class DbConnect
{
    protected $connection;

    // Permet aux autres classes de récupérer $connection sans casser l'encapsulation
    public function getConnection()
    {
        return $this->connection;
    }

    public function __construct()
    {
        try {
            $dsn = 'mysql:host=' . self::SERVER . ';dbname=' . self::DBNAME;
            $this->connection = new PDO($dsn, self::USER, self::PASSWORD);

            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION;
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC;
            PDO::ATTR_EMULATE_PREPARES => false;

        } catch (Exception $e) {
            die('Erreur : ' . $e->getMessage());
        }
    }
}
```

→ Liste des saisons – Models/ClassementsModel.php - getAllSeasonsForSelect()

Cette **méthode**, dont la **classe** hérite de **DbConnect**, récupère la liste des saisons pour alimenter un menu déroulant côté front-end. La **requête** joint la table des **catégories** afin d'obtenir des informations complètes (numéro, catégorie, jeu, plateforme, statut). Les résultats sont **triés** pour distinguer **saisons actives et archivées**, puis retournés sous forme **d'objets** via **fetchAll()**, directement exploitables dans la **vue**. Elle illustre un **accès en lecture avec jointure**.

```
use App\Core\DbConnect;

class ClassementsModel extends DbConnect
{
    // Récupère toutes les saisons actives et inactives pour l'interface
    public static function getAllSeasonsForSelect()
    {
        $db = new DbConnect();
        $sql = "
            SELECT s.id AS season_id, s.season_number, c.name
            FROM seasons s
            JOIN categories c ON c.id = s.category_id
            ORDER BY s.status DESC, c.name ASC, s.season_number ASC
        ";
        return $db->getConnection()->query($sql)->fetchAll();
    }
}
```

→ Classements pilotes de la saison choisie – getDriversStandingsBySeason(\$seasonId)

Cette **méthode** renvoie le classement d'une saison **sélectionnée** par l'utilisateur. Elle reprend la logique du classement actif, en s'appuyant sur une **requête SQL** qui implémente le **calcul** des statistiques et du classement.

L'utilisation d'une **requête préparée** permet de sécuriser le paramètre **season_id** et de garantir une protection contre les **injections SQL**, tout en donnant accès aux saisons archivées.

*(voir annexe p.56
du code de composants
d'accès aux données)*

```
public static function getDriversStandingsBySeason($seasonId)
{
    $db = new DbConnect();
    $sql = "
        SELECT
            ds.*,
            -- Infos saison
            s.videogame,
            s.platform,
            -- Statistiques
            COUNT(DISTINCT gpp.gp_id) AS gp_count,
            COUNT(CASE WHEN gs.pole_position_driver_id = ds.driver_id THEN 1 ELSE 0 END) AS pole_position_driver,
            COUNT(CASE WHEN gs.fastest_lap_driver_id = ds.driver_id THEN 1 ELSE 0 END) AS fastest_lap_driver,
            SUM(gpp.points) AS total_points
        FROM drivers_standings ds
        LEFT JOIN gp_points gpp
            ON gpp.gp_id = ds.gp_id
            AND gpp.driver_id = ds.driver_id
        LEFT JOIN gp_stats gs ON gs.gp_id = ds.gp_id

        WHERE ds.season_id = :season_id
        AND ds.season_status = 'desactive'

        GROUP BY ds.season_id, ds.driver_id
        ORDER BY ds.category ASC, ds.total_points DESC
    ";

    $stmt = $db->getConnection()->prepare($sql);
    $stmt->execute(['season_id' => $seasonId]);
    return $stmt->fetchAll();
}
```

V. Présentation d'éléments de sécurité de l'application

Voici les **mécanismes de sécurité** mis en place au sein de l'application, en complément des composants front-end et back-end déjà détaillés. Dans cette application, j'ai choisi de considérer la sécurité comme un **élément central**. Mon objectif est de protéger les utilisateurs, les données et l'administration, tout en gardant un **code clair** et **maintenable**.

Pour cela, je mets en place plusieurs **mécanismes globaux** : sessions sécurisées, rôles utilisateurs, protections contre les attaques courantes et gestion rigoureuse des formulaires.

→ Session et authentification sécurisées

Dès le cœur de l'application dans **Core/Auth.php**, je configure des **paramètres** de sécurité renforcés. Le mode **strict des sessions** empêche l'utilisation d'identifiants de session non générés par l'application, protégeant ainsi contre les attaques de session. Les **paramètres des cookies** renforcent la sécurité en limitant leur portée, leur durée de vie et leur accessibilité, notamment en imposant l'usage du HTTPS et en empêchant leur transmission lors de requêtes provenant de sites tiers.

```
class Auth
{
    public static function start()
    {
        if (session_status() === PHP_SESSION_NONE) {
            // Sécuriser la session : strict mode et cookies sûrs
            ini_set('session.use_strict_mode', 1);
            // Nom de session personnalisé si plusieurs applis sur
            session_name('team_eracing_sid');

            // Paramètres cookies : à définir AVANT session_start()
            $secure = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') ||
                (!empty($_SERVER['HTTP_X_FORWARDED_PROTO']) && $_SERVER['HTTP_X_FORWARDED_PROTO'] === 'https');

            session_set_cookie_params([
                'lifetime' => 0,
                'path' => '/',
                'domain' => $_SERVER['HTTP_HOST'],
                'secure' => $secure,
                'httponly' => true,
                'samesite' => 'Strict'
            ]);

            session_start();
        }
    }
}
```

Ensuite, j'ai **centralisé** la vérification d'authentification. La méthode **check()** me permet de savoir simplement si un utilisateur est connecté, et **hasRole()** vérifie son rôle. J'ai fait ce choix pour ne pas répéter la même logique partout, et pour pouvoir modifier une règle de sécurité à un seul endroit si nécessaire.

En résumé, je **sécurise** l'accès à l'application dès **la base**, et tous les contrôleurs en bénéficient.

```
// Vérifie si un user est connecté
public static function check(): bool
{
    self::start();
    return isset($_SESSION['user_id']);
}

// Vérifie si l'utilisateur a un rôle spécifique
public static function hasRole(string $requiredRole): bool
{
    self::start();
    return isset($_SESSION['role']) && $_SESSION['role'] === $requiredRole;
}
```

→ Contrôle d'accès avec middleware

Pour protéger les pages sensibles, j'ai mis en place un **middleware d'authentification** dans le **contrôleur principal**. Avant d'exécuter une action, je vérifie systématiquement si l'utilisateur est connecté et s'il possède le rôle requis. S'il n'a pas les droits, je redirige vers la connexion ou j'affiche une erreur 403. J'ai choisi cette approche car aucune fonctionnalité critique ne peut être exécutée sans passer par ce filtre. Cela fonctionne comme un petit "**pare-feu**" interne : simple, lisible et efficace.

Ensuite j'appelle le **middleware** directement dans le **constructeur** de tous les **contrôleurs**. Ici, j'ai choisi de limiter l'accès uniquement aux administrateurs. Si je veux élargir à plusieurs profils, il me suffit de passer un tableau de rôles. Cette approche rend les règles d'accès claires, visibles et faciles à modifier.

```
protected function authMiddleware(string|array $requiredRoles = null)
{
    \App\Core\Auth::start();

    if (!\App\Core\Auth::check()) {
        header('Location: index.php?controller=auth&action=login');
        exit;
    }

    // Vérification des rôles
    if ($requiredRoles) {
        $userHasRole = false;

        if (is_array($requiredRoles)) {
            foreach ($requiredRoles as $role) {
                if (\App\Core\Auth::hasRole($role)) {
                    $userHasRole = true;
                    break;
                }
            }
        } else {
            $userHasRole = \App\Core\Auth::hasRole($requiredRoles);
        }

        if (!$userHasRole) {
            http_response_code(403);
            $content = "<div class='section-dashboard'>
                        <h1>Accès refusé</h1>
                    </div>";

            include dirname(__DIR__) . '/Views/base.php';
            exit;
        }
    }
}
```

```
class UsersController extends Controller {
    public function __construct()
    {
        // Seuls les administrateurs peuvent accéder à ce contrôleur
        $this->authMiddleware("Administrateur");

        // Pour élargir les accès aux autres rôles en array
        // $this->authMiddleware(["Administrateur", "Moderateur", "U
    }
}
```

→ Protection contre les attaques courantes

Avant de traiter une action importante (CRUD), je valide **toujours** les **champs** nécessaires avec la méthode **validatePost** issue de la classe **Form**. Cette règle est simple : je ne fais jamais confiance aux données envoyées par l'utilisateur. Cela évite les

formulaires incomplets, incohérents ou malveillants. Ensuite, pour toutes les requêtes liées à la base de données, j'ai décidé d'utiliser des **requêtes préparées**. Je ne passe **jamais** directement des **valeurs** issues des formulaires dans les **requêtes SQL**. Cette technique bloque efficacement l'injection SQL.

```
// Créer un utilisateur
public function create()
{
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        if (Form::validatePost($_POST, ['email', 'password', 'role_id'])) {
            try {
                $stmt = $pdo->prepare(
                    "INSERT INTO users (email, password_hash, role_id) VALUES (?, ?, ?)");

                if ($stmt->execute([$email, $password_hash, $role_id])) {
                    $message = "Utilisateur créé avec succès";
                    $classMsg = "msg-success";
                }
            }
        }
    }
}
```

→ Sécurité des mots de passe

J'ai choisi d'utiliser **password_hash()** pour **stocker** les mots de passe. Ainsi, aucun mot de passe en clair n'apparaît dans la base. Lors de la connexion, je **vérifie** la valeur avec **password_verify()**. Même en cas de fuite, les mots de passe restent très difficiles à exploiter. Ce choix suit exactement les **bonnes pratiques** modernes en PHP.

```
class UsersController extends Controller {
    // Créer un utilisateur
    public function create()
    {
        // ...
        $password_hash = password_hash($POST['password'], PASSWORD_BCRYPT);
        // ...
    }
}

class AuthController extends Controller
{
    // Traitement du login
    public function login()
    {
        // ...
        if ($user && password_verify($password, $user->password_hash)) {
            // ...
        }
    }
}
```

→ Déconnexion et cycle de vie de session

Lorsqu'un utilisateur se déconnecte, je ne me contente pas de supprimer une variable. Je **vide** la session, je **supprime** le cookie et je **régénère** un nouvel identifiant. J'ai fait cela pour éviter les attaques de **fixation de session** et empêcher la **réutilisation** d'une ancienne session. De cette manière, le cycle de connexion / déconnexion reste **propre** et **sécurisé**.

```
public function logout()
{
    // Utilise gestionnaire centralisé
    \App\Core\Auth::start();
    // Vide toutes les variables
    $_SESSION = [];
    // Supprime le cookie de session
    if (ini_get("session.use_cookies")) {
        $params = session_get_cookie_params();

        setcookie(
            session_name(), '', time() - 42000,
            $params['path'], $params['domain'], $params['secure']
        );
    }
    // Détruit la session
    session_destroy();
    // Régénère un nouvel id vide
    session_regenerate_id(true);
    header('Location: index.php?controller=auth&action=login');
    exit();
}
```

→ Protection CSRF

J'ai choisi de gérer le **CSRF** au niveau du cœur de l'application dans la classe **Auth**. Chaque session possède un **jeton unique** et **aléatoire** qui doit être présent dans chaque formulaire sensible. S'il manque ou est incorrect, la requête est refusée. Cela protège toutes les actions critiques contre les envois frauduleux.

```
// Génère un token CSRF
public static function csrfToken(): string
{
    self::start();
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

// Vérifie un token CSRF
public static function validateCSRF(string $token): bool
{
    self::start();
    return isset($_SESSION['csrf_token'])
        && hash_equals($_SESSION['csrf_token'], $token);
}
```

→ Protection XSS

Enfin, j'ai intégré un **échappement** automatique des formulaires avec **htmlspecialchars** dans la classe **Form** également. Tous les **champs** et **labels** générés sont sécurisés de la même manière. Grâce à cela, je réduis fortement les risques de **XSS** et limite les erreurs humaines.

```
class Form
{
    // Méthode permettant d'ajouter un ou des attributs
    private function addAttributes(array $attributes): string
    {
        $att = "";
        foreach ($attributes as $attribute => $value) {
            // Échapper nom/valeur d'attribut pour éviter injection
            $attrName = htmlspecialchars(
                $attribute, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
            $attrValue = htmlspecialchars(
                (string)$value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
            $att .= " {$attrName}=\"{$attrValue}\"";
        }
        return $att;
    }
}
```

→ Conclusion

Avec ces choix, j'ai construit une **application sécurisée** en :

- Centralisant les règles de sécurité.
- Sécurisant les sessions à la connexion et déconnexion.
- Contrôlant les rôles des utilisateurs et les accès aux contrôleurs.
- Protégeant les formulaires et les requêtes.
- Stockant correctement les mots de passe.
- Mettant en place les protections CSRF et XSS de manière globale.

Cela me permet d'avoir un code plus **propre** et **fiable** dans le temps.

VI. Présentation d'un jeu d'essai élaboré par le candidat de la fonctionnalité la plus représentative (données en entrée, données attendues, données obtenues) et analyse des écarts éventuels

Ce **jeu d'essai** se concentre sur l'ajout des résultats de Grands Prix dans l'application, en prenant en compte une **situation particulière** : un pilote ayant **changé d'équipe** entre deux courses. Cette fonctionnalité est **représentative du projet** car elle combine la gestion des données saisies par l'utilisateur, le calcul automatique des points pilotes et constructeurs, et l'historisation correcte des résultats passés.

→ Fonctionnalité testée

Fonctionnalité : ajout d'un résultat de GP pour un pilote ayant changé d'équipe.

Objectif : vérifier que le classement constructeurs applique correctement les points en fonction de l'équipe sélectionnée à chaque GP, sans retirer de points aux équipes précédemment sélectionnées.

Importance : ce test illustre la logique métier centrale de l'application et la cohérence des calculs sur l'historique des données.

→ Description du jeu d'essai

Le test consiste à :

Vérifier les données actuelles : pilote X a couru le GP précédent avec l'équipe A.

Ajouter le résultat du GP suivant pour ce pilote avec l'équipe B.

Modifier l'association du pilote avec l'équipe A pour l'équipe B.

Contrôler si :

- Le classement pilotes est correct.
- Le classement constructeurs attribue les points à la nouvelle équipe.
- Les points du pilote obtenus avec l'ancienne équipe restent inchangés.

→ Données en entrée

Les entrées à saisir dans le Dashboard Modérateur :

- **Login** : moderateur@gmail.com + mot de passe
- **Table GP - Résultats** : F2 Saison 1 - GP 1 / Cédric A. / Red Bull / position 1 / 25 points
- **Table GP - Résultats** : F2 Saison 1 - GP 2 / Cédric A. / McLaren / position 2 / 18 points
- **Table Associations Pilotes - Teams** : F2 Saison 1 / Cédric A. / McLaren

GP1	GP2	ASSOCIATION
Ajouter résultat	Ajouter résultat	Ajouter association
GP : F2 - Saison 1 / GP 1	GP : F2 - Saison 1 / GP 2	Saison : F2 - Saison 1 (F1 Championship)
Pilote : Cédric A.	Pilote : Cédric A.	Pilote : Cédric A.
Team : Red Bull	Team : McLaren	Équipe : McLaren
Position : 1	Position : 2	
Points : 25	Points : 18	

Contraintes :

Un pilote peut changer d'équipe entre GP.

Les points constructeurs doivent être affectés à l'équipe courante du GP.

Classements pilotes cumulés par pilote.

Les classements constructeurs cumulent les points des pilotes de l'équipe pour chaque GP séparément.

→ Données attendues

Sortie prévue :

- Cédric A : $25 + 18 = 43$ points
- Red Bull : $25 + 0 = 25$ points
- McLaren : $0 + 18 = 18$ points
- Classement Pilotes : Cédric A. / 43 points
- Classement Constructeurs : Red Bull : 25 points / McLaren : 18 points

Comportement attendu :

Les points du GP1 et GP2 sont attribués à Cédric A.

Les points du GP1 vont uniquement à l'équipe Red Bull.

Les points du GP2 vont uniquement à l'équipe McLaren.

Les points du GP1 restent intacts malgré le changement d'équipe du pilote.

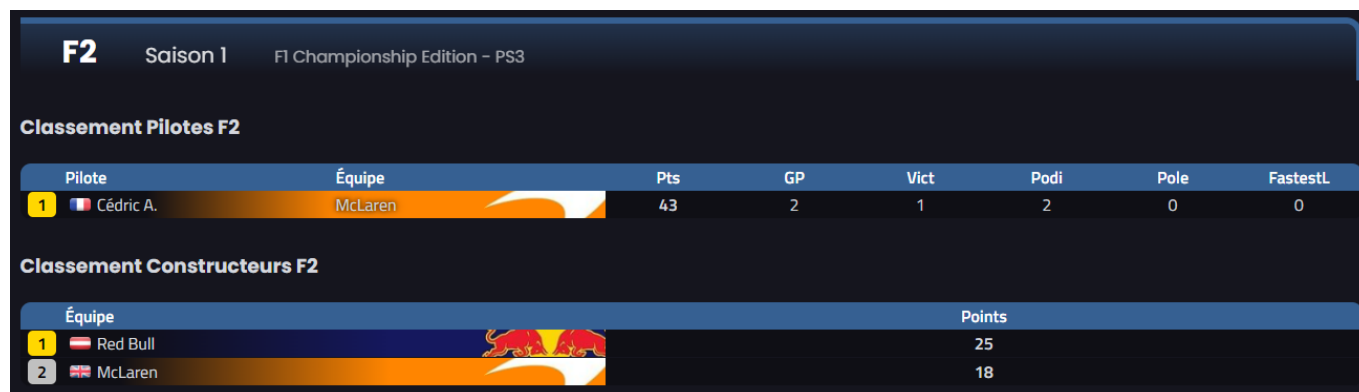
Les classements pilotes et constructeurs reflètent la logique métier correctement.

→ Données obtenues

Après saisie des données en entrée et traitement :

- **Classement Pilotes** : Cédric A. / 43 points
- **Classement Pilotes** : Cédric A. correctement associé à sa nouvelle équipe du GP2 (McLaren)
- **Classement Constructeurs** : Red Bull : 25 points / McLaren : 18 points

Les résultats correspondent exactement aux données attendues :



The screenshot displays the 'F2 Saison 1' interface for the 'F1 Championship Edition - PS3'. It features two tables: 'Classement Pilotes F2' and 'Classement Constructeurs F2'.

	Pilote	Équipe	Pts	GP	Vict	Podi	Pole	FastestL
1	Cédric A.	McLaren	43	2	1	2	0	0

	Équipe	Points
1	Red Bull	25
2	McLaren	18

→ Analyse des écarts

Écart constaté : Aucun.

Analyse : le calcul des classements prend correctement en compte le changement d'équipe pour le GP2, tout en conservant les points précédemment attribués à sa précédente équipe.

Conclusion : cette fonctionnalité respecte les règles métier, la logique de cumul de points est correcte et la gestion historique des équipes est fiable.

→ Lien avec les tests

Ce **jeu d'essai** complète le **tableau des tests fonctionnels manuels (voir annexe p.58)**.

Il se concentre sur un cas transversal, touchant pilote, équipe et classements.

Cela permet de valider le fonctionnement des **calculs automatiques** et **l'intégrité** des historiques.

Il illustre également la cohérence entre la **saisie**, le **traitement métier** et **l'affichage final**.

VII. Description de la veille effectuée durant le projet sur les vulnérabilités de sécurité, description des vulnérabilités éventuellement trouvées et des failles potentiellement corrigées

Au cours du projet, une **veille** a été effectuée autour des problématiques de **sécurité** des applications web. Elle a permis de mettre en évidence des **vulnérabilités** potentielles liées à la gestion des **sessions** utilisateurs, en particulier un dysfonctionnement de la **déconnexion** qui ne détruisait pas complètement la session. Cette analyse a conduit à revoir les mécanismes d'authentification et de déconnexion afin de **renforcer** la protection contre l'usurpation de session et de se conformer aux **bonnes pratiques**.

→ Vulnérabilité de sécurité identifiée durant le projet

Une vulnérabilité a été détectée dans le mécanisme de **déconnexion** utilisateur.

Lors des tests fonctionnels manuels pendant le développement :

Après avoir cliqué sur **Déconnexion**, j'étais correctement renvoyé à la page de connexion mais la **session** n'était pas **détruite**. L'accès au tableau de bord restait possible si je cliquais à nouveau sur **Dashboard**. Il s'agissait donc d'une déconnexion uniquement **visuelle**, la session PHP restant **active** en arrière-plan. Cette faille relevait donc une **mauvaise gestion** des sessions dans la méthode **logout** du **AuthController**.

→ Analyse de l'impact potentiel

Si elle n'avait pas été **corrigée**, cette **faille** aurait pu entraîner :

- **Accès non autorisé** à l'espace administrateur.
- **Divulgateion** d'informations confidentielles.
- **Contournement** de l'authentification.
- **Réutilisation** d'une session sur un poste partagé.
- **Responsabilité** du développeur en cas de fuite.

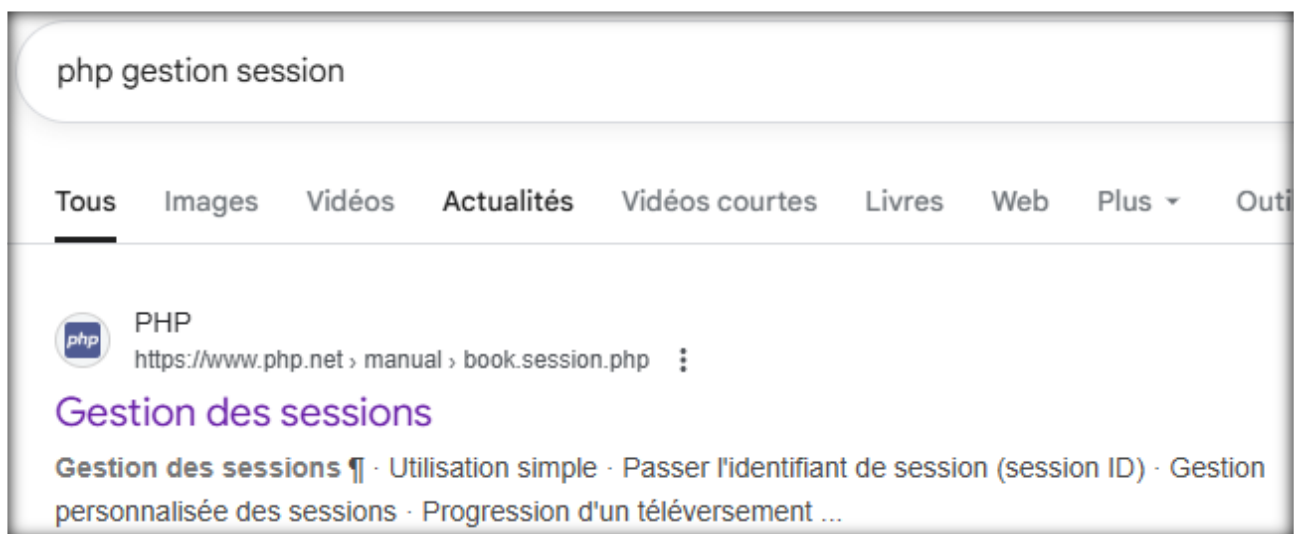
→ Documentations consultées

La veille a été effectuée via :

- **OWASP** qui permet d'identifier, comprendre et prévenir les vulnérabilités de sécurité.
- **Documentation officielle PHP** qui décrit les fonctions et bonnes pratiques.

J'ai notamment utilisé les **mots-clés** suivants :

- php gestion session
- php session_destroy logout
- session management
- session fixation attacks



Le **recoupement** des bonnes pratiques fournies par la **documentation officielle PHP** ainsi que les analyses des vulnérabilités par **OWASP** m'ont permis de valider les **solutions** à mettre en œuvre.

session management

TousImagesVidéosActualitésVidéos courtesLivresWebPlus ▾Outils ▾

OWASP
<https://cheatsheetseries.owasp.org> > ... · [Traduire cette page](#) ·

Session Management - OWASP Cheat Sheet Series

All sessions should implement an absolute timeout, regardless of session activity. This timeout defines the maximum amount of time a session can be active, ...

[Session ID Properties](#)[Session Management...](#)[Cookies](#)[Session Expiration](#)

Session Management Search

OWASP Cheat Sheet Series
REST Security
Ruby on Rails
SAML Security
SQL Injection Prevention
Secrets Management
Secure AI Model Ops
Secure Cloud Architecture
Secure Code Review
Secure Product Design
Securing Cascading Style Sheets
Server Side Request Forgery Prevention
Serverless FaaS Security
Session Management
Software Supply Chain Security
Symfony
TLS Cipher String
Third Party Javascript Management
Third Party Payment Gateway Integration.md
Threat Modeling
Transaction Authorization
Transport Layer Protection
Transport Layer Security

Renew the Session ID After Any Privilege Level Change

The session ID must be renewed or regenerated by the web application after any privilege level change within the associated user session. The most common scenario where the session ID regeneration is mandatory is during the authentication process, as the privilege level of the user changes from the unauthenticated (or anonymous) state to the authenticated state though in some cases still not yet the authorized state. Common scenarios to consider include; password changes, permission changes, or switching from a regular user role to an administrator role within the web application. For all sensitive pages of the web application, any previous session IDs must be ignored, only the current session ID must be assigned to every new request received for the protected resource, and the old or previous session ID must be destroyed.

The most common web development frameworks provide session functions and methods to renew the session ID, such as `request.getSession(true)` & `HttpSession.invalidate()` (J2EE), `Session.Abandon()` & `Response.Cookies.Add(new...)` (ASP .NET), or `session_start()` & `session_regenerate_id(true)` (PHP).

The session ID regeneration is mandatory to prevent session fixation attacks where an attacker sets the session ID on the victim user's web browser instead of gathering the victim's session ID, as in most of the other session-based attacks, and independently of using HTTP or HTTPS. This protection mitigates the impact of other web-based vulnerabilities that can also be used to launch session fixation attacks, such as HTTP response splitting or XSS (see [here](#) and [here](#)).

A complementary recommendation is to use a different session ID or token name (or set of session IDs) pre and post authentication, so that the web application can keep track of anonymous users and authenticated users without the risk of exposing or binding the user session between both states.

→ Corrections apportées dans le code

Suite à ces recherches, la **méthode** de déconnexion a été **revue**. Cela m'a amené à comprendre qu'une simple redirection n'était pas suffisante et qu'il fallait apporter les **modifications recommandées**.

Les **actions** mises en place :

- Vidage des variables de session :
\$_SESSION = []
(empêche la réutilisation locale).
- Suppression du cookie de session :
setcookie()
(empêche le navigateur de renvoyer l'ancienne session).
- Destruction de la session serveur :
session_destroy()
(détruit côté serveur).
- Régénération d'identifiant :
session_regenerate_id(true)
(empêche la fixation et la réutilisation d'un ancien identifiant de session).
- Contrôle d'accès systématique aux pages protégées.

```
Controllers/AuthController.php
66 77      public function logout()
67 78      {
68 -      session_start();
69 +      // Utilise gestionnaire centralisé
70 +      \App\Core\Auth::start();
71 +
72 +      // Vide toutes les variables
73 +      $_SESSION = [];
74 -      unset($_SESSION['csrf_token']);
75 +
76 +      // Supprime le cookie de session
77 +      if (ini_get("session.use_cookies")) {
78 +          $params = session_get_cookie_params();
79 +
80 +          setcookie(
81 +              session_name(),
82 +              '',
83 +              time() - 42000,
84 +              $params['path'],
85 +              $params['domain'],
86 +              $params['secure'],
87 +              $params['httponly']
88 +          );
89 +      }
90 +
91 +      // Détruit la session
92 +      session_destroy();
93 +
94 +      // Régénère un nouvel id vide
95 +      session_regenerate_id(true);
96 +
97 +      header('Location: index.php?controller=auth');
98 +      exit();
```

Le mécanisme de la **méthode logout** est désormais complet, non réutilisable et conforme aux recommandations **OWASP** et **PHP**.

→ Résultat obtenu

Après les corrections et les tests, la session est désormais **réellement détruite**. Toute tentative d'accès forcé à une page protégée via l'URL redirige automatiquement vers la page de connexion, ce qui confirme que la **faille** a été **neutralisée**.

→ Conclusion

Cette démarche m'a montré que la sécurité ne se limite pas aux fonctionnalités visibles : la gestion des sessions est un point **critique** et doit être traitée avec **rigueur**. Cette expérience m'a permis **d'améliorer** la sécurité de l'application et d'adopter de **meilleures pratiques** pour mes futurs développements.

VIII. Déploiement

8.1 Environnements

Le projet est conçu pour fonctionner sur plusieurs **environnements** :

→ Développement (local - Wamp Server)

- Wamp Server (Apache + PHP)
- Base MySQL gérée via phpMyAdmin
- Gestion des versions avec Git et GitHub

Cet environnement m'a permis de **développer** et de **tester** l'ensemble des fonctionnalités.

→ Environnement de test (OVH - CEFii)

- Hébergement fourni par le centre de formation
- Configuration proche de la production
- Base MySQL dédiée
- Accès restreint

Cette étape a été indispensable pour tester le **déploiement**, la **migration** et le bon fonctionnement des fonctionnalités avant diffusion publique.

→ Production (hébergement définitif - O2SWITCH & internet.bs)

- Domaine acheté sur internet.bs avec HTTPS activé : <https://team-eracing.fr>
- Hébergement stable et mutualisé
- Base MySQL avec sauvegardes régulières
- Interface d'administration réellement utilisée

Après le développement et les tests, la **plateforme officielle** sera **accessible au public** et aux organisateurs sur cet hébergement.

8.2 Plan de déploiement

→ Acteurs

- **Développeur** (moi) : préparation des fichiers, base de données et mise en ligne.
- **Formateur** (Olivier) : assistance éventuelle en cas de problème de migration sur OVH.
- **Supports techniques** (OVH & O2SWITCH) : assistance éventuelle en cas de problème serveur.
- **Testeurs** (communauté) : retours d'utilisation et suggestions d'améliorations.

→ Planning

- Phase de développement et tests en local : **Novembre et Décembre 2025**
- Déploiement sur l'environnement de préproduction : **Janvier 2026**
- Mise en production et hébergement final de l'application : **Février 2026**

→ Méthodologie

- Sauvegarde de la base de données et des fichiers existants en local.
- Transfert des fichiers via **FileZilla** vers l'hébergement **OVH / O2SWITCH**.
- Connexion FTP avec identifiants.
- Upload des fichiers du projet dans le dossier racine.

→ Migration de la base MySQL

- Création de la base dédiée racing_championship_db via **phpMyAdmin**.
- Import du **fichier SQL** racing_championship_db_install.sql

→ Configuration :

- Mise à jour des paramètres de connexion dans **Core/DbConnect.php**
- Liaison du nom de domaine **internet.bs** avec le serveur **O2SWITCH**.
- Tests post-déploiement : navigation sur le site, contrôle des fonctionnalités et des accès.

8.3 Plan de déploiement simplifié en local

L'objectif de ce déploiement simplifié est de **permettre** à un développeur de **configurer** rapidement l'application web en **environnement local**, à partir du code disponible sur **GitHub**. Cette procédure facilite la réutilisation des fichiers pour **créer** une application **adaptée** à sa **propre communauté**, tout en respectant les bonnes pratiques de **sécurité**.

Voici les instructions détaillées dans le **README** du dépôt **GitHub** :

- **1.** Cloner le dépôt (ou télécharger le ZIP) :

```
git clone https://github.com/paraguyan99/racing-championship-app.git
```

- **2.** Créer votre Virtual Host relié au dossier du dépôt cloné, par exemple avec Wampserver.
- **3.** Créer la base MySQL via phpMyAdmin et importer le script présent dans le fichier :

```
racing_championship_db_install.sql
```

- **4.** Configurer, si besoin, la connexion à la BDD dans le fichier :

```
Core/DbConnect.php
```

- **5.** Se connecter au Dashboard Administrateur de l'application web avec l'utilisateur par défaut :

identifiant : admin@racing-championship-app.fr

mdp : admin12345

- **6.** Accéder à la table Utilisateurs et créez votre propre utilisateur + mot de passe sécurisé avec le rôle Administrateur.
- **7.** Supprimer l'utilisateur par défaut :

admin@racing-championship-app.fr

8.4 Sauvegarde et sécurité

- Les hébergeurs assurent des **sauvegardes régulières**, permettant une restauration rapide en cas d'incident serveur ou de mauvaise manipulation.
- En complément, des **exports automatiques** de la base de données seront mis en place en local afin de disposer d'une copie hors-site. Cette stratégie "double sauvegarde" limite fortement les risques de perte définitive de données.
- Une **veille** continue sera effectuée sur les **vulnérabilités** et bonnes pratiques de **sécurité**. L'objectif est d'anticiper les risques et de maintenir l'application dans un état sécurisé et stable.

IX. Références

→ **Formula1.com - Site officiel**

Inspiration graphique et organisation des contenus (structure et ambiance visuelle liée au thème F1).

→ **Figma.com**

Outil utilisé pour la réalisation des **zoning**, **wireframes** et **prototype** de l'application.

→ **OWASP.org** (Open Web Application Security Project)

Référence pour identifier, comprendre et prévenir les principales **vulnérabilités de sécurité**.

→ **PHP.net**

Consultation des **fonctions** et **bonnes pratiques** côté serveur.

→ **MySQL.com**

Aide pour la **conception** des tables, l'écriture des **requêtes SQL** et l'optimisation des **relations** entre données.

→ **OpenAI.com**

Assistance pour **reformuler** du code répétitif (Controllers/Models ressemblants) et la recherche de **solutions** front-end.

→ **MDN - developper.mozilla.org** (Mozilla Developer Network)

Documentation pour HTML, CSS et JavaScript.

→ **W3C - w3.org** (World Wide Web Consortium)

Bonnes pratiques liées aux **standards** web, au **responsive design** et à l'**accessibilité**.

→ **CNIL** (Commission nationale de l'informatique et des libertés)

Recherches concernant la gestion des **données personnelles** et les **mentions légales** à appliquer.

X. Conclusion

Ce projet m'a permis de mettre en œuvre l'ensemble des **compétences acquises** durant la **formation**, tout en affrontant de réels défis techniques. Le développement de cette application riche en tableaux et en données m'a obligé à travailler finement le responsive design afin de garantir une lecture claire sur mobile, tablette et desktop. Côté back-end, la veille quotidienne sur les vulnérabilités potentielles et la mise en place de mécanismes de protection m'ont permis de mieux comprendre les enjeux sur la sécurité.

Je suis également **fier** d'avoir conçu cette application de A à Z, sans framework, en développant moi-même la structure MVC, la gestion des routes, l'authentification et l'administration. Cette approche m'a permis de **maîtriser** en profondeur le fonctionnement de l'ensemble du projet.

Aujourd'hui, la plateforme est opérationnelle et sera réellement **utilisée**. Elle continuera **d'évoluer** au rythme des besoins et des retours de notre communauté.

Enfin, je serai également ravi que ce projet puisse servir **d'inspiration** à d'autres développeurs, afin qu'ils puissent l'améliorer et en créer une version personnelle pour leur communauté. Contribuer à ce type de **partage** fait pleinement partie de l'esprit du développement web. Ce serait la plus gratifiante des récompenses.

Je tiens à remercier **Olivier**, mon formateur en développement, qui m'a accompagné tout au long de mon parcours, ainsi que l'ensemble de l'école du **CEFii d'Angers** pour leur soutien et leur encadrement. L'apprentissage à distance a pu être intimidant au début, mais il s'est révélé être un véritable **déclencheur** : il m'a fait découvrir une nouvelle passion, renforcer ma confiance en moi et mon autonomie, et comprendre que cette formation est un **tremplin idéal** vers le monde professionnel, où il faudra régulièrement chercher par soi-même les évolutions et bonnes pratiques du développement.

XI. Annexes comportant les éléments de la fonctionnalité la plus représentative, côté front-end et back-end

11.1 Maquettes des interfaces utilisateur

→ Liens Figma pour accéder aux éléments présentés en page 6 et 8

ZONING



WIREFRAMES



PROTOTYPE



11.2 Captures d'écran des interfaces utilisateur et code correspondant

→ Page d'Accueil

Sur cette page, l'objectif principal est d'optimiser à la fois **l'expérience utilisateur** et le **référencement**. La structure est simplifiée afin de favoriser une lecture rapide sur mobile, avec des sections courtes et hiérarchisées.

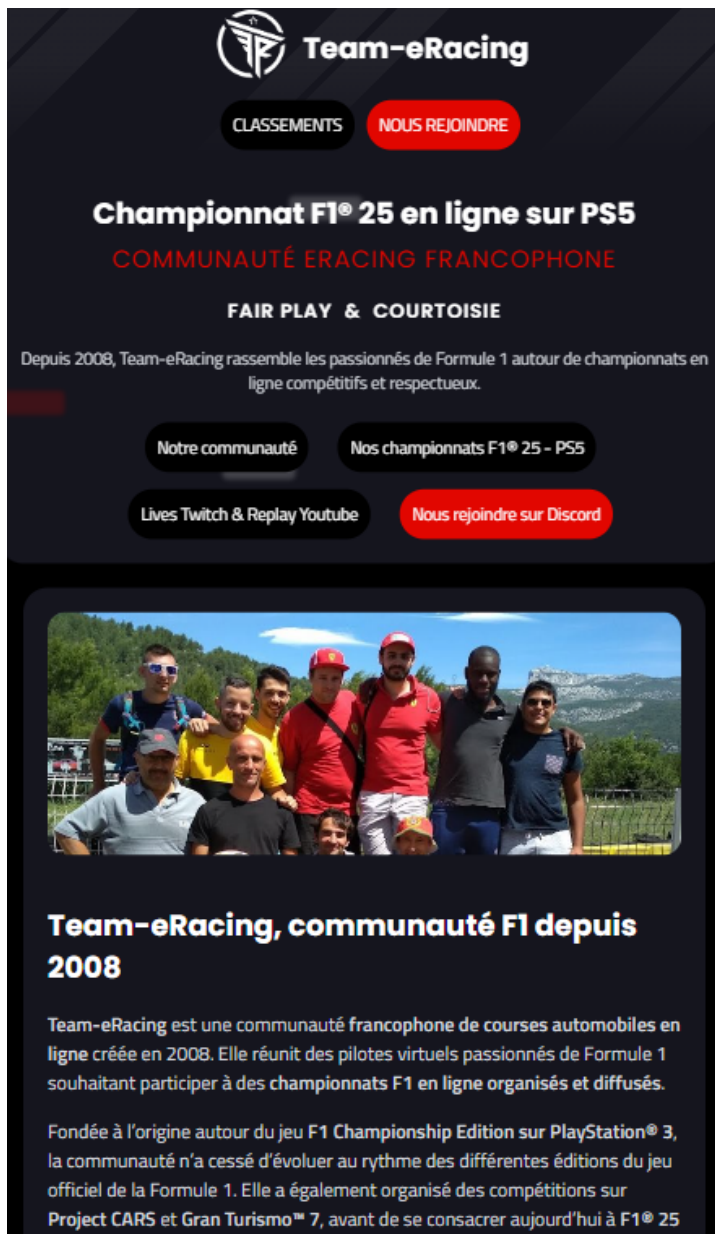
En cours de développement, j'ai ajouté un **bandeau animé** en CSS, absent des maquettes initiales. Ce choix permet de capter l'attention sans alourdir la navigation ni dégrader les performances. Les **balises sémantiques** sont utilisées pour améliorer l'accessibilité et aider les moteurs de recherche à comprendre la structure de la page. Toujours dans une logique d'UX, j'ai intégré un **bouton** "retour en haut de page", commun à l'ensemble du site.

Les captures d'écran sont accompagnées d'extraits de code commentés.

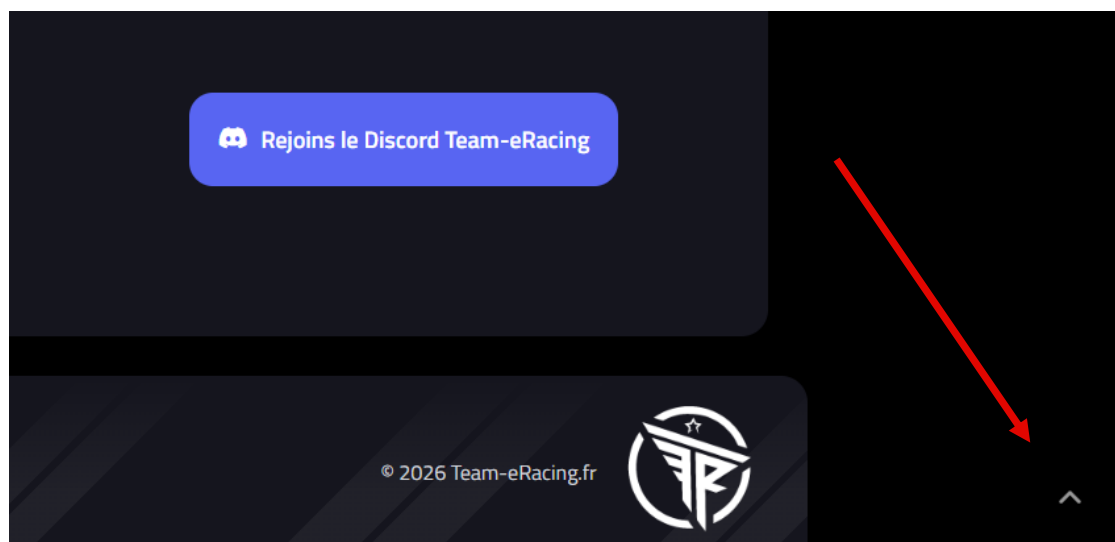
DESKTOP



MOBILE



BOUTON RETOUR EN HAUT DE PAGE



Views/home/index.php

Balises sémantiques h1, h2, p, strong.

```
<div class="hero-content">
  <h1>
    Championnat F1® 25 en ligne sur PS5
    <span class="community-tag">Communauté eRacing francophone</span>
  </h1>

  <section class="hero-section flex-reverse" id="presentation">
    <div class="hero-text-block">
      <h2>Team-eRacing, communauté F1 depuis 2008</h2>
      <p>
        <strong>Team-eRacing</strong> est une communauté <strong>F1</strong>
        créée en 2008. Elle réunit des pilotes virtuels passionnés
        participer à des <strong>championnats F1 en ligne organisés</strong>
      </p>
    </div>
  </section>
</div>
```

HTML du bandeau animé, les div avec classes permet le ciblage de ces zones ensuite en CSS.

```
> home > index.php
You, 16 minutes ago | 1 author (You)
<section class="hero">
  <div class="hero-bg">
    <div class="hero-car"></div>
    <div class="hero-car red rev"></div>
    <div class="hero-car"></div>
    <div class="hero-car red rev"></div>
    <div class="hero-car"></div>
  </div>
</section>
```

public/style.css

En CSS, j'appelle chaque classe de ces div pour définir la couleur, la direction et la vitesse des voitures, afin de varier les mouvements. L'animation est configurée en infinie pour tourner en continu.

```
/* Animation voiture droite->gauche */
.hero-car.rev {
  animation: heroCarMoveRev 6s linear infinite;
}

@keyframes heroCarMoveRev {
  0% { transform: translateX(120vw) translateY(0); }
  100% { transform: translateX(-120vw) translateY(0); }
}

/* Différentes lignes et vitesses */
.hero-bg .hero-car:nth-child(1) { top: 15%; animation-duration: 4s; }
.hero-bg .hero-car:nth-child(2) { top: 30%; animation-duration: 6s; }
.hero-bg .hero-car:nth-child(3) { top: 45%; animation-duration: 5s; }
.hero-bg .hero-car:nth-child(4) { top: 60%; animation-duration: 7s; }
.hero-bg .hero-car:nth-child(5) { top: 75%; animation-duration: 6.5s; }
```

Views/base.php

Ce fichier est commun à toutes les pages du site et évite la duplication du code.

Balise sémantique main affichant dynamiquement le contenu de **home/index.php** grâce à **\$content**.

```
<main>

<!-- Affichage dynamique de la variable $content -->
<?= $content ?>

</main>
```

JavaScript affichant le bouton retour en haut de page avec effet smooth.

```
<!-- Bouton retour en haut de page -->
<button id="scrollToTop" aria-label="Retour en haut de page">
|   <i class="fas fa-chevron-up"></i>
</button>

<script>
document.addEventListener("DOMContentLoaded", function () {
    const scrollBtn = document.getElementById("scrollToTop");

    window.addEventListener("scroll", () => {
        if (window.scrollY > 300) {
            scrollBtn.classList.add("show");
        } else {
            scrollBtn.classList.remove("show");
        }
    });

    scrollBtn.addEventListener("click", () => {
        window.scrollTo({
            top: 0,
            behavior: "smooth"
        });
    });
});
</script>
```

→ Page Classements

L'un des points techniques majeurs a été **l'adaptation** des **tableaux** aux différents formats d'écran. Ils ont été retravaillés pour rester **lisibles** sur **smartphone**, en réorganisant certaines cellules, en compactant les contenus et en ajustant progressivement les tailles de texte et les marges.

Les captures d'écran et extraits de code commentés montrent concrètement comment le CSS et quelques scripts JS gèrent l'affichage dynamique, de façon à conserver des tableaux clairs quelles que soient les dimensions de l'appareil. Ce fonctionnement **responsive** est le même pour les classements constructeurs, les listes des GP et toutes les tables du Dashboard.

DESKTOP

Classements

Dernière mise à jour : 05/01/2026 14:58 / F2 - Saison 1 - GP 2 - Sepang (Malaisie)

Saison 1 - F1 Championship Edition - PS3

F1 Saison 1 F1 Championship Edition - PS3

Classement Pilotes F1

	Pilote	Équipe	Pts	GP	Vict	Podi	Pole	FastestL
1	Jujubiker	Toyota	141	16	8	15	0	0
2	Martio	Williams	68	7	6	7	0	0
3	Guignol81	Ferrari	57	17	0	5	0	0
4	Jimboparigo	Renault	51	14	0	4	0	0
5	Chapi-chapo	Super Aguri	38	9	0	2	0	0
6	Didi511	Honda	29	9	0	1	0	0
7	Senna76	BMW Sauber	23	7	0	1	0	0
8	Fox	McLaren	13	4	0	1	0	0
9	Nordschleife	Red Bull	11	5	0	1	0	0
10	Ludovico6	Toro Rosso	5	4	0	0	0	0
11	Gafit	Spyker	4	9	0	0	0	0

TABLETTE

Classements

Dernière mise à jour : 05/01/2026 14:58 / F2 - Saison 1 - GP 2

Saison 1 - F1 Championship Edition - PS3

F1

Saison 1

F1 Championship Edition - PS3

Classement Pilotes F1

	Pilote	Équipe	Pts	GP	Vict	Podi	PP	FL
1	Jujubiker	Toyota	141	16	8	15	0	0
2	Martlio	Williams	68	7	6	7	0	0
3	Guignol81	Ferrari	57	17	0	5	0	0
4	Jimboparisgo	Renault	51	14	0	4	0	0
5	Chapi-chapo	Super Aguri	38	9	0	2	0	0
6	Didi511	Honda	29	9	0	1	0	0
7	Senna76	BMW Sauber	23	7	0	1	0	0
8	Fox	McLaren	13	4	0	1	0	0
9	Nordschleife	Red Bull	11	5	0	1	0	0
10	Ludovico6	Toro Rosso	5	4	0	0	0	0
11	Gafit	Spyker	4	9	0	0	0	0

MOBILE

Classements

MAJ : 05/01/2026 14:58

Saison 1 - F1 Championship Edition - F

F1

Saison 1

F1 Championship Edition - PS3

Classement Pilotes F1

	Pilote	Equ	Pts	GP	VI	Po	PP	FL
1	Jujubike	Toyota	141	16	8	15	0	0
2	Martlio	Williams	68	7	6	7	0	0
3	Guignol8	Ferrari	57	17	0	5	0	0
4	Jimbopar	Renault	51	14	0	4	0	0
5	Chapi-ch	Super Aguri	38	9	0	2	0	0
6	Didi511	Honda	29	9	0	1	0	0
7	Senna76	BMW Sauber	23	7	0	1	0	0
8	Fox	McLaren	13	4	0	1	0	0
9	Nordschl	Red Bull	11	5	0	1	0	0
10	Ludovico	Toro Rosso	5	4	0	0	0	0
11	Gafit	Spyker	4	9	0	0	0	0

Views/classements/standings.php

HTML des Classements Pilotes avec chaque balise ciblée par une classe.

```
<!-- Classement Pilotes -->
<?php if (!empty($listByCategory[$categoryName])): ?>
    <h3 style="margin-top:30px;">Classement Pilotes <?= htmlspecialchars($categoryName) ?></h3>

    <div class="table-responsive">
        <table class="dashboard-table drivers-table">
            <thead>
                <tr>
                    <th class="badge-width">Position</th>
                    <th>Pilote</th>
                    <th>Équipe</th>
                    <th class="text-center">Points</th>
                    <th class="text-center">GP</th>
                    <th class="text-center">Victoires</th>
                    <th class="text-center">Podiums</th>
                    <th class="text-center">Pole Position</th>
                    <th class="text-center">Fastest Lap</th>
                </tr>
            </thead>
            <tbody>
                <?php $position = 1; ?>
                <?php foreach ($listByCategory[$categoryName] as $row): ?>
                    <!-- Pilote -->
                    <td class="driver-cell"
                        style="--team-color: <?= htmlspecialchars($row->team_color ?? '') ?>
                        ">
                        <div class="driver-gradient"></div>

                        <span class="driver-content">
                            <?php if (!empty($row->driver_flag ?? null)): ?>
                                

                            <!-- Class pour cibler le nom du pilote en JavaScript et en rédu
                            <span class="driver-name">
                                <?= htmlspecialchars($row->nickname) ?>
                            </span>
                        </span>
                    </td>
                </tr>
            </tbody>
        </table>
    </div>
</?>
```

Views/base.php

J'ai choisi de fonctionner avec plusieurs feuilles de styles indépendantes pour améliorer la maintenabilité et la lisibilité du code, et éviter d'avoir un seul fichier CSS trop long et difficile à modifier. J'utilise à l'intérieur de ces fichiers des media queries pour affiner certains détails comme lorsque l'écran fait moins de 500 pixels.

```
<!-- CSS personnel, pas de Framework -->
<link rel="stylesheet" href="style.css" />
<link rel="stylesheet" href="style700px.css" media="screen and (max-width: 700px)" />
<link rel="stylesheet" href="style900px.css" media="screen and (min-width: 701px) and (max-width: 900px)" />
<link rel="stylesheet" href="style1400px.css" media="screen and (min-width: 901px)" />
```

public/style700px.css

Ici, j'appelle donc la feuille de style lorsque l'écran fait moins de 700 pixels de largeur. Cela me permet de redéfinir les en-têtes des tableaux, la taille des polices et des cellules.

```
# style700px-mobilev1.7.css > .drivers-table thead th:nth-child(9)::after

/* Classements */
/* Classement Pilotes uniquement */
.drivers-table thead th:nth-child(1)::after { content: ""; }
.drivers-table thead th:nth-child(2)::after { content: "Pilote"; }
.drivers-table thead th:nth-child(3)::after { content: "Equ"; }
.drivers-table thead th:nth-child(4)::after { content: "Pts"; }
.drivers-table thead th:nth-child(5)::after { content: "GP"; }
.drivers-table thead th:nth-child(6)::after { content: "Vi"; }
.drivers-table thead th:nth-child(7)::after { content: "Po"; }
.drivers-table thead th:nth-child(8)::after { content: "PP"; }
.drivers-table thead th:nth-child(9)::after { content: "FL"; }

.drivers-table thead th {
    font-size: 0;
}

.drivers-table thead th::after {
    font-size: 0.875rem;
}

/* Colonne Pilote */
.drivers-table td.driver-cell,
.drivers-table th:nth-child(2) {
    width: 30%;
}
```

Views/classements/standings.php

J'ai utilisé également du JavaScript pour réduire la taille des noms des pilotes pour que la coupure soit propre plutôt que de masquer sous la cellule suivante. Cette fonction est appelée dès le chargement de la page ainsi qu'au redimensionnement.

```
<!-- Responsive - Réduit taille du nom des pilotes / des écuries
<script>
    function updateResponsiveNames() {
        const w = window.innerWidth;

        /* PILOTES (Classement Pilotes) */
        document.querySelectorAll('.driver-name').forEach(el => {
            if (!el.dataset.fullname) {
                el.dataset.fullname = el.textContent.trim();
            }

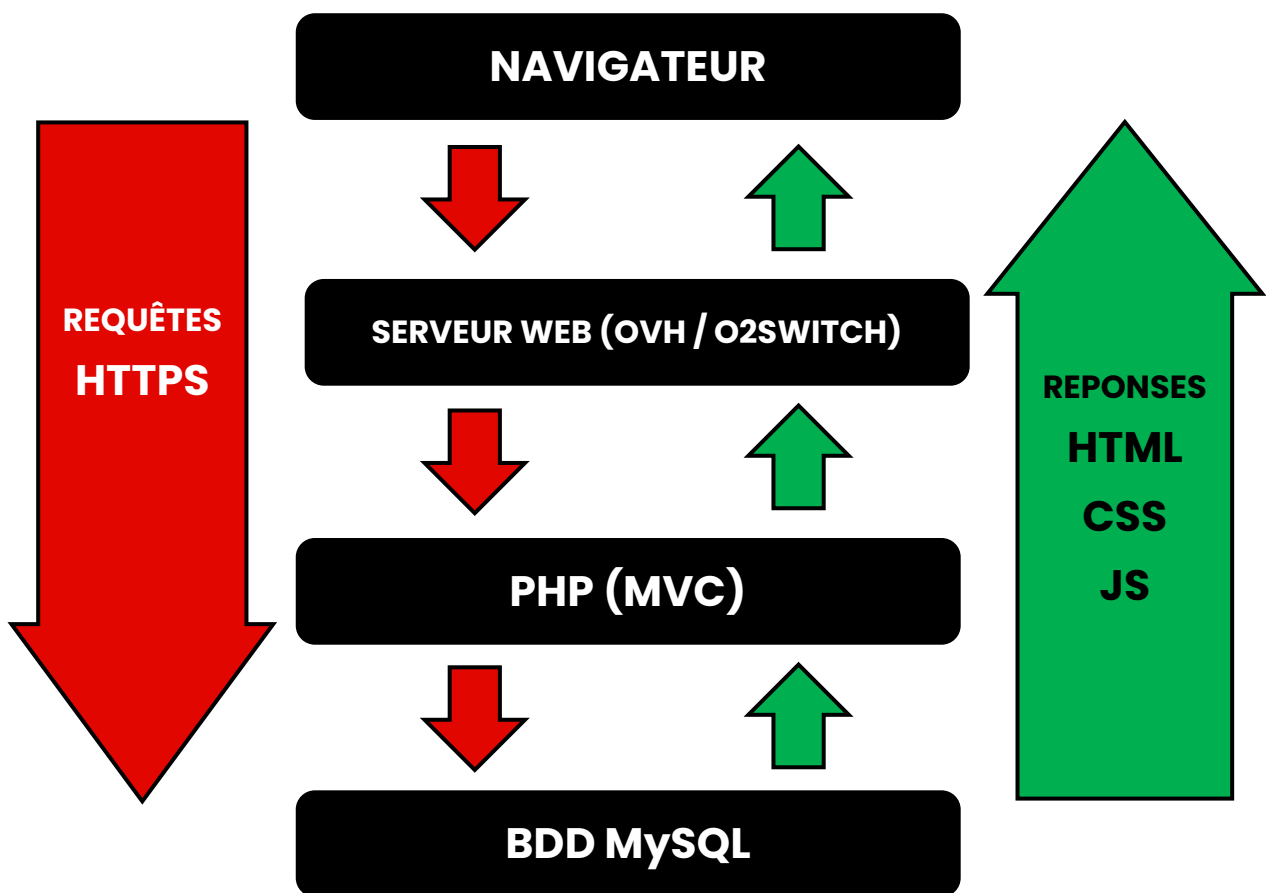
            const full = el.dataset.fullname;

            if (w <= 500) {
                el.textContent = full.substring(0, 8);
            }
            else if (w <= 700) {
                el.textContent = full.substring(0, 12);
            }
            else if (w <= 900) {
                el.textContent = full.substring(0, 16);
            }
            else {
                el.textContent = full.substring(0, 20);
            }
        });
    }
});
```

Ces exemples illustrent la façon dont j'ai pensé, développé et adapté **l'interface** aux différents supports, tout en restant **maintenable** et performante.

11.3 Schéma d'architecture du modèle MVC

Ce **schéma** correspond à l'architecture réelle de l'**application** : le navigateur envoie une requête sécurisée au serveur de l'hébergeur, le contrôleur PHP (MVC) traite la demande, interroge la base de données MySQL, puis renvoie au navigateur une vue générée en HTML/CSS/JS.



11.4 Code des composants métier les plus significatifs

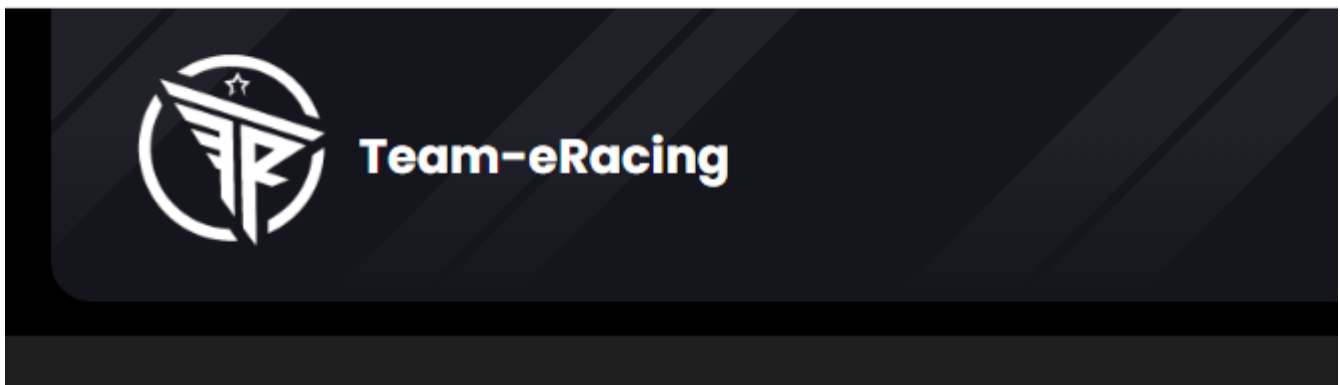
Contexte

Afin de présenter les différentes parties de code, je vais m'appuyer sur deux exemples concrets mettant en œuvre les **composants métier** ainsi que la **logique de sécurité centralisée** utilisée dans l'ensemble des contrôleurs. Cela permettra de comprendre précisément le **parcours** effectué par l'application au sein de mon modèle MVC.

Deux scénarios sont présentés :

- Utilisation d'une URL par un modérateur afin d'accéder au **CRUD** de la gestion des pilotes (dont l'accès est réservé aux Administrateurs).

```
racing-championship-app/public/index.php?controller=drivers&action=index
```



- Modification d'un token CSRF dans un formulaire POST pour **mettre à jour** un pilote.

```
<div class="form-wrapper">
  <form action="index.php?controller=drivers&action=update&id=1"
    method="POST" class="login-form">
...
    <input type="hidden" name="csrf_token" value="df591b6be2d4632802
      6a29acf839e3ef8cb7627c4d317d8cc922d6dc6207b80"> == $0
    <div class="form-group">...</div>
```

Core/Auth.php

Au cœur de l'application, cette classe permet, via la méthode **start()**, de sécuriser la session utilisateur et de contrôler la durée de connexion. Les autres méthodes permettent de vérifier si l'utilisateur est connecté, de récupérer et vérifier son rôle, ainsi que de générer et de comparer le token CSRF.

```
class Auth
{
    public static function start()
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();

            // Deconnexion de l'utilisateur et destruction de la session après 5 m
            $lifetime = 300 ; // durée en secondes 300 = 5 minutes

            if (isset($_SESSION['time_activity']) && (time() - $_SESSION['time_act
                session_unset(); // supprime les variables
                session_destroy(); // détruit la session
                header("Location: index.php?controller=auth&action=login");
                exit();
            }
        }

        // Vérifie si un user est connecté
        public static function check(): bool
        {
            self::start();
            return isset($_SESSION['user_id']);
        }

        // Récupère le rôle (string)
        public static function role(): ?string
        {
            self::start();
            return $_SESSION['role'] ?? null;
        }

        // Vérifie si l'utilisateur a un rôle spécifique
        public static function hasRole(string $requiredRole): bool
        {
            self::start();
            return isset($_SESSION['role']) && $_SESSION['role'] === $requ

        }

        // Génère un token CSRF
        public static function csrfToken(): string
        {
            self::start();
            if (empty($_SESSION['csrf_token'])) {
                $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
            }
            return $_SESSION['csrf_token'];
        }

        // Vérifie un token CSRF
        public static function validateCSRF(string $token): bool
        {
            self::start();
            return isset($_SESSION['csrf_token'])
                && hash_equals($_SESSION['csrf_token'], $token);
        }
    }
}
```

Controllers/Controller.php

Il s'agit ici du contrôleur parent dont héritent l'ensemble des contrôleurs de l'application.

La méthode **authMiddleware()** est chargée d'appeler la classe **Auth** afin d'effectuer l'ensemble des contrôles de sécurité. Cette méthode attend en paramètre le rôle de l'utilisateur, fourni sous forme de chaîne de caractères ou de tableau, puis exécute successivement les contrôles via :

Auth::**check()**, Auth::**hasRole()** et Auth::**validateCSRF()**.

```
use App\Core\Auth;

abstract class Controller
{
    // Verification à chaque appel des Controllers :
    // LOGIN + SESSION + TIMEACTIVITY 5 MIN + ROLES + FAILLES CSRF TOKEN
    protected function authMiddleware(string|array $requiredRoles = null)
    {
        Auth::start();

        if (!Auth::check()) {
            header('Location: index.php?controller=auth&action=login');
            exit;
        }

        // Vérification des rôles
        if ($requiredRoles) {
            $userHasRole = false;

            if (is_array($requiredRoles)) {
                foreach ($requiredRoles as $role) {
                    if (Auth::hasRole($role)) {
                        $userHasRole = true;
                        break;
                    }
                }
            } else {
                $userHasRole = Auth::hasRole($requiredRoles);
            }

            if (!$userHasRole) {
                http_response_code(403);
                // Contenu affichant le message d'erreur
                $content = "<div class='section-dashboard'>
                    <h1>Accès refusé</h1>
                    <p>Vous n'avez pas les autorisations nécessaires.</p>
                    <a class='nav-btn-dashboard' href='index.php'>Retour à l'accueil</a>
                </div>";
                include dirname(__DIR__) . '/Views/base.php';
                exit;
            }
        }

        // Vérification CSRF pour POST
        if ($_SERVER["REQUEST_METHOD"] === "POST") {
            if (!isset($_POST['csrf_token'])
                || !Auth::validateCSRF($_POST['csrf_token'])) {
                http_response_code(403);
                // Contenu affichant le message d'erreur
                $content = "<div class='section-dashboard'>
                    <h1>Token invalide !</h1>
                    <p>Veuillez réessayer ou recharger la page.</p>
                    <a class='nav-btn-dashboard' href='index.php'>Retour à l'accueil</a>
                </div>";
                include dirname(__DIR__) . '/Views/base.php';
                exit;
            }
        }
    }

    public function render(string $path, array $data = [])
    {
    }
}
```

Controllers/DriversController.php

Afin de vérifier le rôle de l'utilisateur et d'autoriser l'accès aux fonctionnalités, chaque contrôleur du Dashboard définit les rôles autorisés via un constructeur appelé avant toute autre méthode. Celui-ci déclenche la méthode **authMiddleware()** du contrôleur parent.

Dans le cas du **DriversController**, le rôle est fourni au format chaîne de caractères.

```
class DriversController extends Controller {  
  
    public function __construct()  
    {  
        // Seuls les administrateurs peuvent accéder à ce controller  
        $this->authMiddleware("Administrateur");  
    }  
}
```

À titre d'exemple, le **GpController** utilise quant à lui un tableau de rôles.

```
class GpController extends Controller {  
  
    public function __construct()  
    {  
        // Accès autorisé aux Admin et Modo  
        $this->authMiddleware(["Administrateur", "Moderateur"]);  
    }  
}
```

Core/Form.php

Concernant le token CSRF, celui-ci est ajouté grâce à la classe Form et à sa méthode **addCSRF()**, présente dans le cœur de l'application. Les autres méthodes de cette classe permettent également de sécuriser de manière centralisée l'ensemble de la construction des formulaires.

```
class Form  
{  
    // Méthode qui ajoute un CSRF TOKEN dans les formulaires  
    public function addCSRF(): self  
    {  
        $token = \App\Core\Auth::csrfToken();  
        $this->formElements .= "<input type='hidden' name='csrf_token' value='"  
            . htmlspecialchars($token, ENT_QUOTES, 'UTF-8') . "'>";  
        return $this;  
    }  
}
```

Controllers/DriversController.php

Pour revenir au **DriversController**, son constructeur appelle la méthode **authMiddleware()**, qui vérifie notamment le rôle de l'utilisateur et la validité du token CSRF. Ces contrôles sont effectués avant toute utilisation d'une méthode de cette classe. La logique métier appliquée dans la méthode **update()** peut alors être observée. Dans un premier temps, une requête **SELECT** est préparée puis exécutée à partir de l'identifiant transmis via la méthode **GET** afin de récupérer les données du pilote et d'hydrater le formulaire. Si une requête **POST** est transmise, une requête **UPDATE** est ensuite préparée puis exécutée dans un bloc **try / catch**, à condition qu'aucune erreur ne soit détectée. L'envoi de cette requête **POST** est rendu possible grâce à la construction du formulaire via la classe **Form**, dont la méthode **addCSRF()** permet l'ajout du token de sécurité.

```
// Mettre à jour un pilote
public function update($id)
{
    // Requete préparée
    $stmt = $pdo->prepare("SELECT * FROM drivers WHERE id=?");
    $stmt->execute([$id]);
    $driver = $stmt->fetch();

    if (!$driver) {
        $message = "Pilote introuvable";
    }

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {

        if (Form::validatePost($_POST, ['nickname', 'country_id', 'status'])) {

            try {
                // Requete préparée
                $stmt = $pdo->prepare("
                    UPDATE drivers
                    SET nickname=?, country_id=?, status=?
                    WHERE id=?
                ");

                if ($stmt->execute([$name, $_POST['country_id'], $_POST['status'], $id])) {
                    $message = "Mise à jour réussie";
                    $classMsg = "msg-success";
                } else {
                    $message = "Erreur lors de la mise à jour";
                    $classMsg = "msg-error";
                }
            } catch (\PDOException $e) {

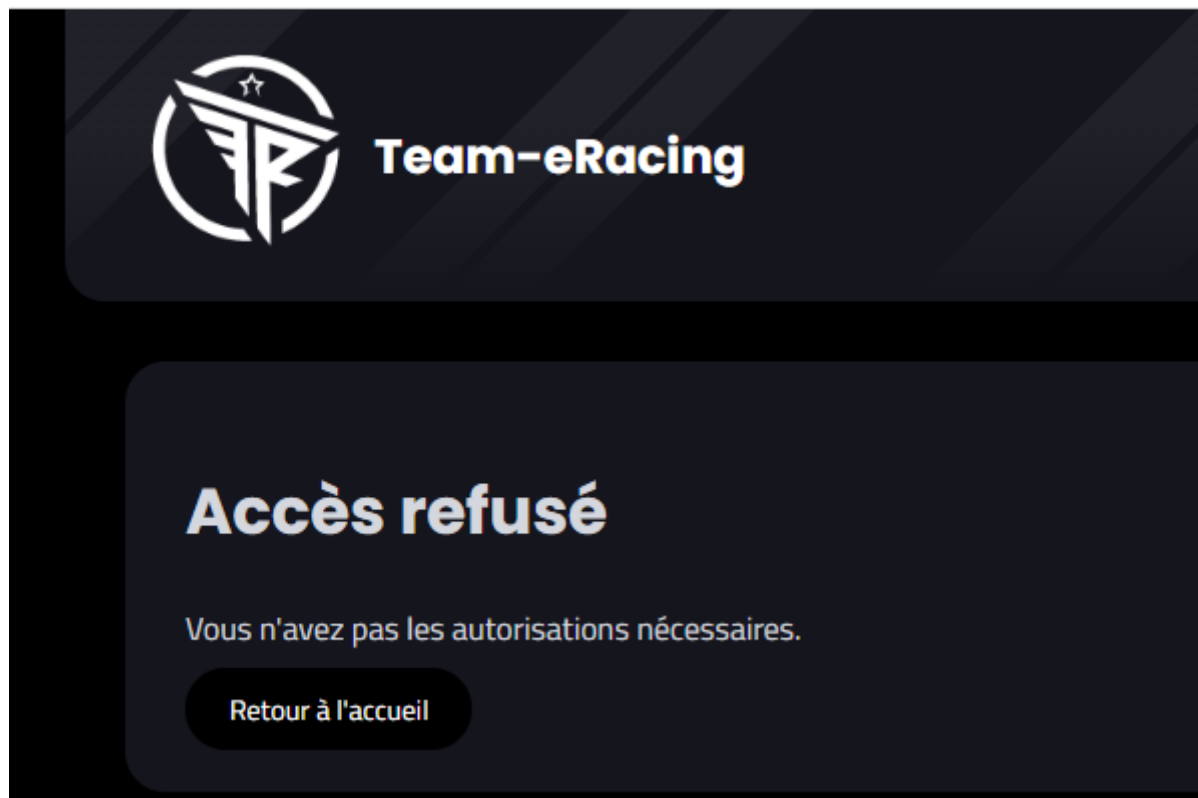
                if ($e->getCode() == 23000) {
                    $message = "Erreur : ce pseudo existe déjà !";
                } else {
                    $message = "Erreur lors de la mise à jour";
                }
            }
        }
    }

    $form = new Form();
    $form->startForm("index.php?controller=drivers&action=update&id=" . $driver->id, "POST")
        ->addCSRF()
        ->addLabel("nickname", "Pseudo :")
        ->addInput("text", "nickname", ["value" => $driver->nickname])
        ->endForm();

    $this->render('dashboard/drivers/update', [
        'list' => DriversModel::all(),
        'countries' => $countries,
        'form' => $form,
        'message' => $message,
        'classMsg' => $classMsg
    ]);
}
```

Voici le résultat obtenu à partir de ces deux scénarios. Ils montrent que la **logique de sécurité centralisée** est systématiquement appliquée avant toute exécution de la **logique métier**.

racings-championship-app/public/index.php?controller=drivers&action=index



Le constructeur de la classe **DriversController** transmet le rôle Administrateur au contrôleur parent, qui effectue la vérification via sa méthode **authMiddleware()** et refuse ainsi l'accès.

```
if (!$userHasRole) {
    http_response_code(403);
    // Titre de la page
    $title = "Team-eRacing";

    // Contenu affichant le message d'erreur
    $content = "<div class='section-dashboard'>
        <h1>Accès refusé</h1>
        <p>Vous n'avez pas les autorisations nécessaires.</p>
        <a class='nav-btn-dashboard' href='index.php'>Retour à l'accueil</a>
    </div>";

    include dirname(__DIR__) . '/Views/base.php';
    exit;
}
```



Le token CSRF du formulaire de la méthode **update()** du **DriversController** a été modifié à l'aide de l'inspecteur. Il est ensuite comparé grâce à la méthode **validateCSRF()**, utilisée dans la méthode **authMiddleware()**, ce qui entraîne le refus de l'exécution de la requête **UPDATE**.

```
// Vérification CSRF pour POST
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    if (!isset($_POST['csrf_token'])
        || !Auth::validateCSRF($_POST['csrf_token'])) {
        http_response_code(403);
        // Titre de la page
        $title = "Team-eRacing";

        // Contenu affichant le message d'erreur
        $content = "<div class='section-dashboard'>
                    <h1>Token invalide !</h1>
                    <p>Veuillez réessayer ou recharger la page.</p>
                    <a class='nav-btn-dashboard' href='index.php'>Retour à l'accueil</a>
                </div>";

        include dirname(__DIR__) . '/Views/base.php';
        exit;
    }
}
```

11.5 Code des composants d'accès aux données les plus significatifs

La classe **DbConnect** sert de point d'accès principal à la base de données. Elle crée une connexion fiable avec **PDO** et applique les réglages nécessaires pour travailler proprement et en sécurité. J'ai choisi **PDO** parce qu'il est **natif** en **PHP**, largement documenté et stable. Les **requêtes préparées** protègent contre les injections SQL, le mode « erreur par exception » facilite le débogage, et l'encodage **utf8mb4** garantit une bonne gestion des caractères. La méthode **getConnection()** me permet de centraliser l'accès à la connexion sans violer l'encapsulation en exposant les détails internes.

```
use PDO;
use Exception;

class DbConnect
{
    protected $connection;
    protected $request;

    // Permet aux autres classes de récupérer $connection sans violer l'encapsulation.
    public function getConnection()
    {
        return $this->connection;
    }

    private const SERVER = 'localhost';
    private const USER = 'root';
    private const PASSWORD = '';
    private const BASE = 'racing_championship_db';

    public function __construct()
    {
        try {
            // Durcir PDO / charset – Core/DbConnect.php
            // Dans le constructeur, améliore le DSN et quelques attributs PDO :
            // charset=utf8mb4 évite problèmes d'encodage (et failles potentielles).
            // ATTR_EMULATE_PREPARES = false améliore la sécurité des requêtes préparées.

            $dsn = 'mysql:host=' . self::SERVER . ';dbname=' . self::BASE . ';charset=utf8mb4';
            $this->connection = new PDO($dsn, self::USER, self::PASSWORD, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_OBJ,
                PDO::ATTR_EMULATE_PREPARES => false // forcer prepares natifs
            ]);
        } catch (Exception $e) {
            die('Erreur : ' . $e->getMessage());
        }
    }
}
```

La méthode **getDriversStandingsBySeason()** calcule le classement des pilotes pour une saison donnée. Elle s'appuie sur plusieurs **tables** liées (categories, seasons, countries, drivers, teams, gp, gp_points, gp_stats) et utilise des **jointures**, **COUNT** et des conditions **CASE WHEN** pour produire directement en base les statistiques nécessaires. La requête préparée **PDO** sécurise l'accès contre les injections SQL. J'ai placé ces **algorithmes** dans les **models** afin de centraliser la **logique métier** liée aux données : le contrôleur reste léger, la vue n'effectue aucun calcul, et la base de données fait le travail le plus efficacement possible. Toutes les requêtes CRUD de l'application suivent la même approche.

```
public static function getDriversStandingsBySeason($seasonId)
{
    $db = new DbConnect();
    $sql = "
        SELECT
            ds.*,

            -- Infos saison
            s.videogame,
            s.platform,

            -- Couleur de la catégorie
            cat.color AS category_color,

            -- Flag pilote
            country.flag AS driver_flag,

            -- Equipe
            t.logo AS team_logo,
            t.color AS team_color,

            -- Statistiques
            COUNT(DISTINCT gpp.gp_id) AS gp_count,
            COUNT(CASE WHEN gs.pole_position_driver = ds.driver_id THEN 1 END) AS pole_count,
            COUNT(CASE WHEN gs.fastest_lap_driver = ds.driver_id THEN 1 END) AS fastestlap_count

        FROM drivers_standings ds

        -- Saison
        JOIN seasons s ON s.id = ds.season_id

        -- Categories (pour color)
        JOIN categories cat ON cat.id = s.category_id

        -- Pilote + flag
        LEFT JOIN drivers d ON d.id = ds.driver_id
        LEFT JOIN countries country ON country.id = d.country_id

        -- Equipe
        LEFT JOIN teams_drivers td
            ON td.driver_id = ds.driver_id
            AND td.season_id = ds.season_id
        LEFT JOIN teams t ON t.id = td.team_id

        -- GP + stats
        JOIN gp g ON g.season_id = ds.season_id

        LEFT JOIN gp_points gpp
            ON gpp.gp_id = g.id
            AND gpp.driver_id = ds.driver_id

        LEFT JOIN gp_stats gs ON gs.gp_id = g.id

        WHERE ds.season_id = :season_id
        AND ds.season_status = 'desactive'

        GROUP BY ds.season_id, ds.driver_id
        ORDER BY ds.category ASC, ds.total_points DESC
    ";

    $stmt = $db->getConnection()->prepare($sql);
    $stmt->execute(['season_id' => $seasonId]);
    return $stmt->fetchAll();
}
```

11.6 Tableau des tests fonctionnels manuels

Le projet étant développé en **MVC sans framework**, la mise en place de **tests unitaires automatisés** aurait nécessité l'ajout d'outils externes et une architecture différente. Pour un projet de cette taille, j'ai privilégié des **tests fonctionnels manuels**, plus pertinents pour valider les fonctionnalités métier.

→ Tests fonctionnels manuels – Tests de sécurité – Tests d'interface

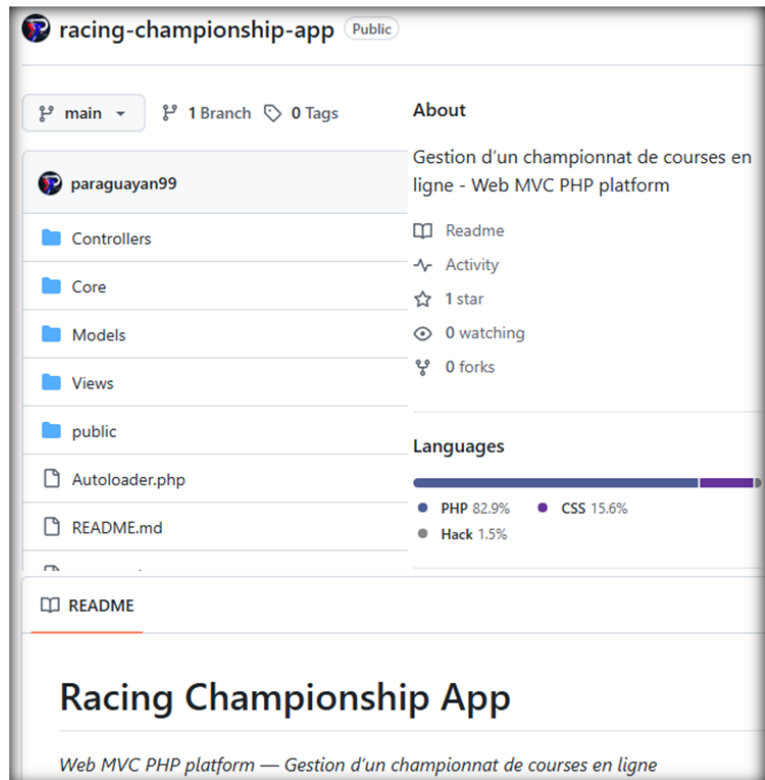
Test	Cas de test	Etapes à suivre	Données d'entrée	Résultats	Commentaires
T001-001 Tests fonctionnels (Functional Testing)	CT001 Authentification	1/ Cliquer sur Dashboard (footer). 2/ Entrer email et MDP. 3/ Cliquer sur Se connecter.	1/ Login correct. 2/ Login incorrect.	1/ Redirigé sur le dashboard administrateur avec affichage du profil connecté. 2/ Message d'erreur : Identifiants invalides en restant sur le formulaire de connexion.	Fonctionnel L'accès au contenu (via URL du Dashboard) est impossible si l'authentification n'est pas correcte.
T001-002	CT002 Déconnexion	1/ Se connecter au Dashboard. 2/ Copier l'URL. 3/ Se déconnecter. 4/ Tenter d'accéder à cette page du Dashboard via l'URL	1/ Login correct 2/ URL copiée pour forcer l'accès au Dashboard : index.php?controller=dashboard&action=index	Utilisateur bien déconnecté et la redirection vers la page de connexion Login s'effectue automatiquement.	Fonctionnel Aspect sécuritaire : La session est bien détruite
T001-003	CT003 Gestion des rôles Administrateur Modérateur Utilisateur	1/ Cliquer sur Dashboard. 2/ Se connecter en Utilisateur. 3/ Tester plusieurs URL du CRUD. 4/ Se connecter en Modérateur. 5/ Tester plusieurs URL du CRUD.	1/ ...?controller=users&action=update&id=1 2/ ... ?controller=countries&action=index 3/ url d'une saison désactivée par l'admin ... ?controller=teamsdrivers&action=delete&id=7	Utilisateur : 1/ 2/ 3/ Accès refusé. Modérateur : 1/ 2/ Accès refusé 3/ Message d'erreur en rouge : la saison est désactivée.	Fonctionnel L'accès aux tables est sécurisé et les restrictions sont appliquées côté serveur : Controller / BDD.
T001-004	CT004 CRUD : Affichage des INDEX du Dashboard	1/ Se connecter en Admin. 2/ Aller dans chacune des pages index des 13 tables de la BDD.	Login correct.	Toutes les tables de la BDD sont visibles sur chacun des index du modèle MVC.	Fonctionnel
T001-005	CT005 Voir les résultats détaillés d'un GP dans le Dashboard	1/ Se connecter. 2/ Aller dans « GP - Résultats ». 3/ Cliquer sur l'icône arrow down pour dérouler les résultats.	Login correct.	Le premier clic déroule les résultats et montre les icônes UPDATE et DELETE. Le deuxième clic referme correctement les résultats.	Fonctionnel
T001-006	CT006 CRUD : Tester toutes les fonctions CREATE	1/ Se connecter. 2/ Aller dans chaque index. 3/ Cliquer sur le bouton « Ajouter ». 4/ Ajouter données attendues et cliquer sur Créer. 5/ Refaire le test en oubliant des données attendues et cliquer sur Créer.	1/ Login correct. 2/ Remplir les formulaires et cliquer sur Créer. 3/ Remplir partiellement les mêmes formulaires en oubliant les données dans des input qui ne doivent pas être NULL.	2/ Toutes les créations ont été effectué et les retours aux index fonctionnent avec un message de succès. 3/ Le message d'erreur en rouge s'affiche correctement et explique pourquoi la création n'a pas été effectué.	Fonctionnel
T001-007	CT007 CRUD : Tester toutes les fonctions UPDATE	1/ Se connecter. 2/ Aller dans chaque index. 3/ Cliquer sur l'icône stylo. 4/ Modifier données et cliquer sur Mettre à jour. 5/ Refaire le test en créant des erreurs de saisies volontaires ou en laissant des champs vides et cliquer sur Mettre à jour.	1/ Login correct. 2/ Données modifiées partiellement sans créer d'erreur. 3/ Deuxième modification en créant une erreur de syntaxe, un chiffre négatif, un chiffre qui ne respecte pas les décimales attendues, un champ vide et enfin des tests de doublons.	2/ Toutes les tables retournent un message de succès de mise à jour. 3/ Chaque erreur a provoqué l'abandon de la mise à jour soit par un contrôle JavaScript ou par un rejet de la requête par le Controller / BDD. Dans ces 2 derniers cas, un message personnalisé en rouge permet de comprendre l'erreur produite.	Fonctionnel

T001-008	CT008 CRUD : Tester toutes les fonctions DELETE	1/ Se connecter. 2/ Aller dans chaque index. 3/ Cliquez sur l'icône corbeille. 4/ Cliquer sur ANNULER. 5/ Cliquer sur l'icône corbeille. 6/ Cliquer sur SUPPRIMER.	Login correct.	4/ Suppression non effectuée et retour à l'index correct pour toutes les tables. 6/ Suppression effectuée et retour à l'index avec un message de succès.	Fonctionnel Toutes les tables empêchent la suppression si une clé étrangère est présente dans une autre table. Aucune suppression en cascade n'a été effectuée.
T002-001 Tests de sécurité (Security Testing)	CT001 Injection SQL	1/ Aller à la page Classements. 2/ Puis sur Circuits. 3/ Choisir un circuit. 4/ Modifier la fin de l'URL avec ces injections SQL et actualiser : OR 1=1 / -- / ' 5/ Se connecter au Dashboard. 6/ Insérer injection SQL dans le formulaire UPDATE de la table teams à la place du nom.	5/ ...&action=index&circuit_id=17 OR 1=1 5/ ...&action=index&circuit_id=17-- 5/ ...&action=index&circuit_id=17' 7/ table teams = ' OR 1=1 --	5/ La requête est sécurisée, aucune donnée non autorisée n'est retournée. L'ID reste 17 et l'injection est ignorée. 7/ Pour l'insertion en BDD, le nom a bien été sauvegardé comme texte sans provoquer d'erreur.	Protégé contre les injections SQL
T002-002	CT002 Faille XSS	1/ Aller à la page Classements. 2/ Puis sur une saison archivée. 3/ Modifier la fin de l'URL en testant ces failles XSS. 4/ Se connecter au Dashboard 5/ Insérer faille XSS dans le formulaire UPDATE de la table teams à la place du nom.	...standings&season_id= 3/ <script>alert('XSS')</script> 3/ 3/ <script>console.log("XSS")</script> 5/ <script>alert('XSS STOCKÉE')</script>	3/ Script non exécuté que ce soit côté navigateur ou dans la console. 5/ Le script est bien enregistré comme texte à la place du nom du pilote dans la BDD mais rien ne s'est exécuté côté navigateur.	Absence de faille XSS
T002-003	CT003 Faille CSRF	1/ Se connecter au Dashboard. 2/ Aller au formulaire UPDATE d'un driver. 3/ Modifier le nom du pilote. 4/ Ouvrir l'inspecteur Chrome et modifier d'un chiffre le token qui se trouve dans l'input hidden.	Modifier un chiffre du token dans : <input type="hidden" name="csrf_token" value="6fa44...260">	Message d'erreur qui s'affiche : Token invalide ! Veuillez réessayer ou recharger la page. Et la mise à jour du nom du pilote n'a pas été effectué.	Absence de faille CSRF
T003-001 Tests d'interface (UI)	CT001 Responsive Mobile / Tablette	1/ Clic droit de la souris et utiliser l'inspecteur Chrome. 2/ Jouer avec les largeurs d'écran pour constater le comportement responsive des pages du site.	Pages visitées et testées : Accueil Classements + Palmarès + Circuits Dashboard : Tables GP Résultats et Teams (avec images des logos)	Affichage responsive de tous les éléments : tableaux / images / textes longs ... de 400px à 1400px	Responsive Mobile Tablet Desktop

11.7 Liens vers GitHub et le site déployé sur OVH

→ Dépôt GitHub :

<https://github.com/paraguyan99/racing-championship-app>



→ Site déployé sur OVH avant la mise en ligne définitive sur 02SWITCH en Février 2026 :

<https://www.cefii-developpements.fr/cedric1493/racing-championship-app/public/>

